二项式系数

$$\sum_{i=k}^{n}\binom{i}{k}=\binom{n+1}{k+1}$$

$$\binom{-n}{k}=(-1)^k\binom{n+k-1}{k}$$

$$\sum_{i=0}^{k}(-1)^i\binom{n}{i}=(-1)^k\binom{n-1}{k}$$

$$\binom{n}{m}\equiv\binom{n\bmod p}{m\bmod p}\binom{\lfloor n/p\rfloor}{\lfloor m/p\rfloor}\pmod p$$

形式一 (对称形式)：

$$f(n)=\sum_{i=0}^{n}(-1)^i\binom{n}{i}g(i)\iff g(n)=\sum_{i=0}^{n}(-1)^i\binom{n}{i}f(i)$$

形式二 (常用常数求和)：

$$f(n)=\sum_{i=0}^{n}\binom{n}{i}g(i)\iff g(n)=\sum_{i=0}^{n}(-1)^{n-i}\binom{n}{i}f(i)$$

形式三 (至少与恰好，极其常用)：

设 $f(k)$ 为“至少包含 k 个”的方案数，$g(k)$ 为“恰好包含 k 个”的方案数：

$$f(k)=\sum_{i=k}^{n}\binom{i}{k}g(i)\iff g(k)=\sum_{i=k}^{n}(-1)^{i-k}\binom{i}{k}f(i)$$

牛顿迭代式

求$F(x)$使得$G(F(x))\equiv 0\pmod{x^n}$
$G(F_0(x))\equiv 0\pmod{x^{\lceil n/2\rceil}}$

$$F(x)\equiv F_0(x)-\frac{G(F_0(x))}{G'(F_0(x))}\pmod{x^n}$$

斯特林数

$\begin{bmatrix}n\\k\end{bmatrix}$ 表示将 n 个不同元素划分为 k 个**非空圆排列**（环）的方案数。

$$\begin{bmatrix}n\\k\end{bmatrix}=\begin{bmatrix}n-1\\k-1\end{bmatrix}+(n-1)\begin{bmatrix}n-1\\k\end{bmatrix}$$

边界条件： $\begin{bmatrix}n\\0\end{bmatrix}=[n=0]$，$\begin{bmatrix}0\\0\end{bmatrix}=1$。

$$x^{\overline{n}} = \prod_{i=0}^{n-1}(x+i) = \sum_{k=0}^{n} \begin{bmatrix} n \\ k \end{bmatrix} x^k$$

$$x^{\underline{n}} = \prod_{i=0}^{n-1}(x-i) = \sum_{k=0}^{n} (-1)^{n-k} \begin{bmatrix} n \\ k \end{bmatrix} x^k$$

有符号第一类斯特林数 $s(n,k) = (-1)^{n-k} \begin{bmatrix} n \\ k \end{bmatrix}$。

第 k 列的指数生成函数：

$$\sum_{n=k}^{\infty} \begin{bmatrix} n \\ k \end{bmatrix} \frac{x^n}{n!} = \frac{(-\ln(1-x))^k}{k!}$$

$\begin{Bmatrix} n \\ k \end{Bmatrix}$ 表示将 n 个不同元素划分为 k 个**无差别非空集合**的方案数。

$$\begin{Bmatrix} n \\ k \end{Bmatrix} = \begin{Bmatrix} n-1 \\ k-1 \end{Bmatrix} + k \begin{Bmatrix} n-1 \\ k \end{Bmatrix}$$

$$\begin{Bmatrix} n \\ k \end{Bmatrix} = \begin{Bmatrix} n-1 \\ k-1 \end{Bmatrix} + k \begin{Bmatrix} n-1 \\ k \end{Bmatrix}$$

$$\begin{Bmatrix} n \\ k \end{Bmatrix} = \frac{1}{k!} \sum_{i=0}^{k} (-1)^i \binom{k}{i} (k-i)^n = \sum_{i=0}^{k} \frac{(-1)^i}{i!} \frac{(k-i)^n}{(k-i)!}$$

$$x^n = \sum_{k=0}^{n} \begin{Bmatrix} n \\ k \end{Bmatrix} x^{\underline{k}}$$

因为 $x^{\underline{k}} = k! \binom{x}{k}$，所以 $x^n = \sum_{k=0}^{n} \begin{Bmatrix} n \\ k \end{Bmatrix} k! \binom{x}{k}$。

$$\sum_{n=k}^{\infty} \begin{Bmatrix} n \\ k \end{Bmatrix} \frac{x^n}{n!} = \frac{(e^x-1)^k}{k!}$$

$$f_n = \sum_{k=0}^{n} \begin{Bmatrix} n \\ k \end{Bmatrix} g_k \iff g_n = \sum_{k=0}^{n} (-1)^{n-k} \begin{bmatrix} n \\ k \end{bmatrix} f_k$$

$$B_n = \sum_{k=0}^{n} \begin{Bmatrix} n \\ k \end{Bmatrix}$$

贝尔数的 EGF 为 e^{e^x-1}

$$\sum_{k=0}^{n} \begin{bmatrix} n \\ k \end{bmatrix} = n!$$

n 个不同的球，放入 k 个不同的盒子，允许为空： 总方案数为 k^n。

n 个不同的球，放入 k 个不同的盒子，不允许为空： 总方案数为 $k! \begin{Bmatrix} n \\ k \end{Bmatrix}$。

$k^n = \sum_{i=0}^{k} \binom{k}{i} i! \begin{Bmatrix} n \\ i \end{Bmatrix}$

$$\sum_{i=0}^{m} i^n = \sum_{i=0}^{m} \sum_{k=0}^{n} \begin{Bmatrix} n \\ k \end{Bmatrix} k! \binom{i}{k} = \sum_{k=0}^{n} \begin{Bmatrix} n \\ k \end{Bmatrix} k! \sum_{i=0}^{m} \binom{i}{k} = \sum_{k=0}^{n} \begin{Bmatrix} n \\ k \end{Bmatrix} k! \binom{m+1}{k+1}$$

拉格朗日反演

$F(0) = 0, F'(0) \neq 0.$
$F(G(x)) = x$ 且 $G(F(x)) = x$

$$[x^n]G(x) = \frac{1}{n}[x^{n-1}] \left(\frac{x}{F(x)} \right)^n$$

$$[x^n]H(G(x)) = \frac{1}{n}[x^{n-1}]H'(x) \left(\frac{x}{F(x)} \right)^n$$

$$[x^m]H(G(x)) = [x^m]H(x) \left(\frac{x}{F(x)} \right)^{m+1} \left(1 - \frac{xF'(x)}{F(x)} \right)$$

$$[x^n]f(x)^k = \frac{k}{n}[x^{n-k}] \left(\frac{x}{g(x)} \right)^n$$

分圆多项式

$$x^n - 1 = \prod_{d|n} \Phi_d(x)$$

$$\Phi_n(x) = \prod_{d|n} (x^d - 1)^{\mu(\frac{n}{d})}$$

Burnside 引理

计算在群 G 作用下，集合 X 中本质不同的元素个数（等价类个数）。

$$|X/G| = \frac{1}{|G|}\sum_{g\in G}|X^g|$$

(本质不同方案数 = 所有置换下“不动点个数”的平均值)

Pólya 定理

Burnside 引理在染色问题上的特化，化简了求不动点的过程。
若有 m 种颜色，置换 g 可以分解为 $c(g)$ 个不相交的循环，则该置换下的不动点个数为 $m^{c(g)}$：

$$|X/G| = \frac{1}{|G|}\sum_{g\in G} m^{c(g)}$$

欧拉变换

处理**无标号物品**的组合问题（例如整数分拆、无根树计数、多重集组合）。已知单个物品的 OGF 为 $A(x)$，求由这些物品组成的无序多重集（物品可重复选，无顺序）的 OGF $E(x)$。

$$E(x) = \prod_{i=1}^{\infty}\left(\frac{1}{1-x^i}\right)^{a_i}$$

$$\ln E(x) = \sum_{i=1}^{\infty}\frac{1}{i}A(x^i)$$

指数型生成函数 (EGF) 的 exp 意义

处理**有标号物品**的组合问题（例如连通图与一般图、森林与树）。
已知“单个连通块/基础组件”的有标号方案数的 EGF 为 $F(x)$，那么由若干个这样的基础组件随意拼装成的“全集”的有标号方案数 EGF $G(x)$ 满足：

$$G(x) = \exp(F(x)) \iff F(x) = \ln(G(x))$$

欧拉函数与五边形数定理

极速求出整数分拆数 $P(n)$，或者处理无穷乘积 $\prod(1-x^k)$。

$$\prod_{k=1}^{\infty}(1-x^k) = \sum_{k=-\infty}^{\infty}(-1)^k x^{k(3k\pm1)/2}$$

矩阵树定理

求任意无向图/有向图的生成树个数。
度数矩阵 D：对角线为顶点的度数（有向图分入度树和出度树）。
邻接矩阵 A：$A_{i,j}$ 表示 i 到 j 的边数。

基尔霍夫矩阵 (拉普拉斯矩阵) L：$L=D-A$。
图的生成树个数等于矩阵 L 划掉任意第 i 行和第 i 列后的主子式的行列式

凯莱公式

完全图的生成树计数（矩阵树定理的特例）。
n 个有标号顶点的无根树数量为 n^{n-2}
n 个有标号顶点的有根树数量为 n^{n-1}
含有 k 个连通块，且这 k 个连通块大小分别为 $s_1,s_2,\ldots,s_k$ 的生成树个数为 $n^{k-2}\prod_{i=1}^{k}s_i$

Bostan-Mori 算法思想

常系数齐次线性递推。已知递推式，求第 n 项（n 可以高达 10^{18}）
所有的常系数线性递推都可以写成有理分式 $F(x)=\frac{P(x)}{Q(x)}$ 的形式，要求 $[x^n]F(x)$。
将分子分母同乘 $Q(-x)$：

$$\frac{P(x)}{Q(x)}=\frac{P(x)Q(-x)}{Q(x)Q(-x)}$$

分母 $Q(x)Q(-x)$ 是一个偶函数，它只含有偶数次项，即可以写成 $V(x^2)$ 的形式。
此时，要提取结果的奇数项或偶数项，只需要提取分子 $P(x)Q(-x)$ 对应的奇/偶次项，然后将问题规模缩小一半（除以 $V(x)$），递归求解。

一些便捷计算

```
int p[N],q[N],r[N],w[N],invnum[N],invfac[N],fac[N],len[N];
int add(int x,int y){return x+y>mod?x+y-mod:x+y;}
int dec(int x,int y){return x-y<0?x-y+mod:x-y;}
int ext(int n){return n==1?1:(1<<(std::__lg(n-1)+1));}
int qpow(int a,int b){int res=1;for (;b;b>>=1,a=1ll*a*a%mod)if (b&1)res=1ll*res*a%mod;return res;}
void get_r(int n){for (int i=0;i<n;i++)r[i]=(r[i>>1]>>1)|(i&1?n>>1:0);}
void print(int *a,int n){for (int i=0;i<n;i++)printf("%d%c",a[i]," \n"[i==n-1]);}
void cpy(int *a,int *b,int n){for (int i=0;i<n;i++)b[i]=a[i];}
```

初始化

```
void init_poly(){
    for(int k=2,m=1;k<=N;k<<=1,m<<=1){
        w[m]=1;
        for(int i=m+1,x=qpow(3,(mod-1)/k);i<k;i++) w[i]=1ll*w[i-1]*x%mod;
    }invnum[1]=1;
    for (int i=2;i<N;i++)invnum[i]=1ll*(mod-mod/i)*invnum[mod%i]%mod;
    invfac[0]=1;
    for (int i=1;i<N;i++)invfac[i]=1ll*invfac[i-1]*invnum[i]%mod;
    fac[0]=1;
    for (int i=1;i<N;i++)fac[i]=1ll*fac[i-1]*i%mod;
}
```

NTT

```
void NTT(int *a,int n,int op){
    for (int i=0;i<n;i++)if (i<r[i])std::swap(a[i],a[r[i]]);
    for (int k=2,m=1;k<=n;m<<=1,k<<=1){
        for (int i=0;i<n;i+=k)
            for (int j=i,*x=w+m;j<i+m;x++,j++){
                int v=1ll*a[j+m]*(*x)%mod;
                a[j+m]=dec(a[j],v);a[j]=add(a[j],v);
            }
    }
    if (op==-1){
        std::reverse(a+1,a+n);
        for (int i=0,v=qpow(n,mod-2);i<n;i++)a[i]=1ll*a[i]*v%mod;
    }
}
```

普通乘法

注意n,m为多项式长度（最高次数加一），不是最高次数吗，结果存在a

```
void mul(int *a,int *b,int n,int m){
    if (!n||!m)return;
    int len=ext(n+m-1);
    for (int i=0;i<len;i++)p[i]=i<n?a[i]:0,q[i]=i<m?b[i]:0;
    get_r(len);
    NTT(p,len,1);NTT(q,len,1);
    for (int i=0;i<len;i++)a[i]=1ll*p[i]*q[i]%mod;
    NTT(a,len,-1);
}
```

不变乘法

结果保留在另外的c数组

```
void mul_2(int *a,int *b,int *c,int n,int m){//数的个数
    if (!n||!m)return;
    int len=ext(n+m-1);
    for (int i=0;i<len;i++)p[i]=i<n?a[i]:0,q[i]=i<m?b[i]:0;
    get_r(len);
    NTT(p,len,1);NTT(q,len,1);
    for (int i=0;i<len;i++)c[i]=1ll*p[i]*q[i]%mod;
    NTT(c,len,-1);
}
```

求逆元

倍增求逆元

$$B(x)\equiv B_0(x)+B_0(x)-A(x)B_0^2(x)\equiv B_0(x)(2-A(x)B_0(x))\pmod{x^n}$$

```
void _inv(int *a,int *b,int n){
    static int c[N],d[N];
    n=ext(n);memset(b,0,4*n);b[0]=qpow(a[0],mod-2);
    for (int k=2;k<=n;k<<=1){
        for (int i=0;i<k<<1;i++)c[i]=i<k?a[i]:0,d[i]=i<k>>1?b[i]:0;
        get_r(k);NTT(d,k,1);
        for (int i=0;i<k;i++)d[i]=1ll*d[i]*d[i]%mod;
        NTT(d,k,-1);get_r(k<<1);NTT(c,k<<1,1);NTT(d,k<<1,1);
        for (int i=0;i<k<<1;i++)c[i]=1ll*c[i]*d[i]%mod;
        NTT(c,k<<1,-1);
        for (int i=0;i<k;i++)b[i]=(2ll*b[i]-c[i]+mod)%mod;
    }
}
```

求导与积分

注意求导时n是被导函数的项数，积分是结果的项数

```
void _diff(int *a,int *b,int n){//a-n个数
    for (int i=1;i<n;i++)b[i-1]=1ll*i*a[i]%mod;b[n-1]=0;
}
void _integ(int *a,int *b,int n){//b-n个数
    for (int i=1;i<n;i++)b[i]=1ll*invnum[i]*a[i-1]%mod;b[0]=0;
}
```

求ln

$$B(x)\equiv\int\frac{A'(x)}{A(x)}dx\pmod{x^n}$$

```
void _ln(int *a,int *b,int n){
    static int c[N],d[N];
    assert(a[0]==1);
    n=ext(n);
    _inv(a,c,n),_diff(a,d,n),mul(c,d,n,n);_integ(c,b,n);
}
```

求exp

$$B(x)\equiv B_0(x)(1-\ln(B_0(x))+A(x))\pmod{x^n}$$

```
void _exp(int *a,int *b,int n){
    static int c[N];
    assert(a[0]==0);
    n=ext(n);memset(b,0,4*n);memset(c,0,4*n);b[0]=1;
    for (int k=2;k<=n;k<<=1){
        _ln(b,c,k);
        for (int i=0;i<k;i++)c[i]=dec(a[i],c[i]);
        c[0]++;mul(b,c,k,k);

    }
}
```

开根

$$B(x)\equiv B_0(x)-\frac{B_0^2(x)-A(x)}{2B_0(x)}\pmod{x^n}$$

```
void _sqrt(int *a,int *b,int n){
    static int c[N],d[N];
    static const int inv2=(mod+1)>>1;
    assert(a[0]==1);
    n=ext(n);memset(b,0,4*n);b[0]=1;
    for (int k=2;k<=n;k<<=1){
        memcpy(c,a,4*k);_inv(b,d,k),mul(c,d,k,k);
        for (int i=0;i<k;i++)b[i]=1ll*(b[i]+c[i])*inv2%mod;
    }
}
```

快速幂

首项不为1与为1的处理方法

```
void _pow(int *a,int *b,int n,string k){
    static int c[N],d[N];
    int u=0,k1=0,k2=0;
    memset(b,0,4*n);
    for (int i=0;i<k.size();i++)k1=(10ll*k1+k[i]-'0')%mod,k2=(10ll*k2+k[i]-'0')%(mod-1);
    for (u=0;u<n&&!a[u];u++);
    if ((u&&k.size()>=5)||1ll*u*k1>=n)return;
    for (int i=u,x=qpow(a[u],mod-2);i<n;i++)c[i-u]=1ll*a[i]*x%mod;
    _ln(c,d,n-u);
    for (int i=0;i<n-u;i++)d[i]=1ll*d[i]*k1%mod;
    _exp(d,c,n-u);
    for (int i=u*k1,x=qpow(a[u],k2);i<n;i++)b[i]=1ll*c[i-u*k1]*x%mod;
}
void _pow_p(int *a,int *b,int n,string k){
    static int c[N],d[N];
    int u=0,k1=0,k2=0;
    for (int i=0;i<k.size();i++)k1=(10ll*k1+k[i]-'0')%mod;
    _ln(a,d,n);
    for (int i=0;i<n;i++)d[i]=1ll*d[i]*k1%mod;
    _exp(d,b,n);
}
```

多项式除法

$a = b \times q + r$

```
void _div(int *a,int *b,int *q,int *r,int n,int m){
    static int c[N],d[N],e[N];
    int len=n-m+1;
    for (int i=0;i<len;i++)c[i]=a[n-1-i];
    for (int i=0;i<len;i++)d[i]=i<m?b[m-1-i]:0;
    _inv(d,e,len);mul(c,e,len,len),std::reverse(c,c+len);
    memcpy(q,c,4*len),mul(c,b,len,m);
    for (int i=0;i<m;i++)r[i]=dec(a[i],c[i]);
}
```

分治

```
void cdq(int l,int r,int *f,int *g){//g已知
    static int c[N];
    if (l==r)return ;
    int mid=(l+r)>>1;
    cdq(l,mid,f,g);mul_2(f+l,g+1,c,mid-l+1,r-l);
    for (int i=mid+1;i<=r;i++)f[i]=add(f[i],c[i-l-1]);
    cdq(mid+1,r,f,g);
}
void cdq2(int l,int r,int maxn,int *f,int *g){//g未知
    static int c[N],d[N];
    if (l==r){
        if (l!=1)f[l]=1ll*f[l]*invnum[l-1]%mod;
        for (int i=l;i<=maxn;i+=l)g[i]=add(g[i],1ll*l*f[l]%mod);
        return;
    }
    int mid=(l+r)>>1;
    cdq2(l,mid,maxn,f,g);
    if (l==1){
        mul_2(f,g,c,mid+1,mid+1);//mid+mid>=r
        for (int i=mid+1;i<=r;i++)f[i]=add(f[i],c[i]);
    }
    else{
        mul_2(f+l,g+1,c,mid-l+1,r-l);//f[l,mid]*g[1,r-l]
        mul_2(f+1,g+l,d,r-l,mid-l+1);//f[1,r-l]*f[l,mid]
        for (int i=mid+1;i<=r;i++)f[i]=add(f[i],add(c[i-l-1],d[i-l-1]));
    }
    cdq2(mid+1,r,maxn,f,g);
}
```

二项式反演

```
void fanyan_2(int *a,int *b,int n){//个数
    static int c[N],d[N];
    for (int i=0;i<n;i++)c[i]=1ll*a[i]*fac[i]%mod*(i%2==0?1:mod-1)%mod;
    for (int i=0;i<n;i++)d[i]=invfac[i];
    std::reverse(c,c+n);
    mul_2(c,d,b,n,n);
    std::reverse(b,b+n);
    for (int i=0;i<n;i++)b[i]=1ll*b[i]*invfac[i]%mod*(i%2==0?1:mod-1)%mod;
}
```

分治乘法

用于处理多个$(1+kx)$相乘

```
#define ls(x) (x<<1)
#define rs(x) (x<<1|1)
    int _c[19][N],_d[19][N];
    void mul_solve(int l,int r,int u,const int*a,int *b,int dep){
        if (l==r){b[0]=1;b[1]=mod-a[l];len[u]=1;return;}//初始化
        int mid=l+r>>1;
        static int f[N],g[N];
        mul_solve(l,mid,ls(u),a,f,dep+1);cpy(f,_c[dep],len[ls(u)]+1);
        mul_solve(mid+1,r,rs(u),a,g,dep+1);cpy(g,_d[dep],len[rs(u)]+1);
        len[u]=len[ls(u)]+len[rs(u)];
        mul_2(_c[dep],_d[dep],b,len[ls(u)]+1,len[rs(u)]+1);
    }
```

用于处理不同次数的分治乘法
m是多项式的个数，init_p是各个多项式的首地址，init_deg是度数，ans存放结果

```
struct PolyInfo {
    int* p;
    int deg;
    bool operator<(const PolyInfo& other) const {
        return deg > other.deg; // 小根堆，度数小的优先合并
    }
};
int _merge_pool[N<<2], _merge_top; // 专属静态内存池
int huffman_merge(int m, int **init_p, int *init_deg, int *ans) {
    if (!m) { ans[0] = 1; return 0; }
    std::priority_queue<PolyInfo> pq;
    _merge_top = 0;
    for (int i=1;i<=m;i++) pq.push({init_p[i], init_deg[i]});
    while (pq.size()>1) {
        PolyInfo a=pq.top();pq.pop();
        PolyInfo b=pq.top();pq.pop();
        PolyInfo c={ _merge_pool + _merge_top, a.deg + b.deg };
        _merge_top += c.deg + 1;
        mul_2(a.p, b.p, c.p, a.deg + 1, b.deg + 1);
        pq.push(c);
    }
    PolyInfo res=pq.top(); pq.pop();
    cpy(res.p, ans, res.deg + 1);
    return res.deg;
}
```

多项式平移

$$f(x+c)=\sum_{i=0}^{n}a_i\sum_{j=0}^{i}\binom{i}{j}c^{i-j}x^j=\sum_{j=0}^{n}x^j\frac{1}{j!}\sum_{i=j}^{n}(a_ii!)\frac{c^{i-j}}{(i-j)!}$$

```
void _shift(int *f,int *g,int c,int n) {
    static int A[N], B[N];
    for (int i=0;i<n;i++) A[i]=1ll*f[i]*fac[i]%mod;
    for (int i=0,pc=1;i<n;i++,pc=1ll*pc*c%mod)B[i]=1ll*pc*invfac[i]%mod;
    std::reverse(A,A+n);
    mul_2(A,B,g,n,n);
    std::reverse(g,g+n);
    for (int i=0;i<n;i++) g[i]=1ll*g[i]*invfac[i]%mod;
}
```

第一类斯特林数 · 行 (固定 n)

$$x^{\overline{n}} = \prod_{i=0}^{n-1}(x+i) = \sum_{k=0}^{n} \begin{bmatrix} n \\ k \end{bmatrix} x^k$$

```
void get_S1_row(int n,int *ans) {
    if (!n) {ans[0]=1;return;}
    if (n==1) {ans[0]=0;ans[1]=1;return;}
    static int tmp[N];
    if(n&1){
        get_S1_row(n-1,ans);
        // ans=ans*(x+n-1)
        for (int i=n;i>=1;i--) ans[i]=add(ans[i-1],1ll*ans[i]*(n-1)%mod);
        ans[0]=1ll*ans[0]*(n-1)%mod;
    }else{
        get_S1_row(n>>1,ans);
        _shift(ans,tmp,n>>1,(n>>1)+1);
        mul_2(ans,tmp,ans,(n>>1)+1,(n>>1)+1);
    }
}
```

第二类斯特林数 · 行 (固定 n)

$$\begin{Bmatrix} n \\ k \end{Bmatrix} = \sum_{i=0}^{k} \frac{(-1)^i}{i!} \frac{(k-i)^n}{(k-i)!}$$

```
void get_S2_row(int n,int *ans) {
    static int c[N],d[N];
    for (int i=0;i<=n;i++) {
        c[i]=(i&1)?dec(0,invfac[i]):invfac[i];
        d[i]=1ll*qpow(i,n)*invfac[i]%mod;
    }
    mul_2(c,d,ans,n+1,n+1);
}
```

第一类斯特林数 · 列 (固定 k)

$$\sum_{n=0}^{\infty} \begin{bmatrix} n \\ k \end{bmatrix} \frac{x^n}{n!} = \frac{(-\ln(1-x))^k}{k!}$$

$A(x) = \frac{-\ln(1-x)}{x} = 1 + \frac{x}{2} + \frac{x^2}{3} + \dots$
原式化为$x^k \frac{A(x)^k}{k!}$

```
// 求解无符号第一类斯特林数第 k 列，求前 n 项，存入 ans
void get_S1_col(int n,int k,int *ans) {
    if (k>=n) {memset(ans,0,4 * n);return;}
    static int A[N],B[N],C[N];
    memset(A,0,4*(n-k));
    for (int i=0;i<n-k;i++) A[i]=invnum[i+1];
    _ln(A,B,n-k);
    for (int i=0;i<n-k;i++) B[i]=1ll*B[i]*k%mod;
    _exp(B,C,n-k);
    memset(ans,0,4*n);
    int inv_k=invfac[k];
    for(int i=k;i<n;i++){
        ans[i]=1ll*C[i-k]*inv_k%mod*fac[i]%mod;
    }
}
```

第二类斯特林数 · 列 (固定 k)

$$\sum_{n=0}^{\infty} \begin{Bmatrix} n \\ k \end{Bmatrix} \frac{x^n}{n!} = \frac{(e^x-1)^k}{k!}$$

$A(x) = \frac{e^x-1}{x} = 1 + \frac{x}{2!} + \frac{x^2}{3!} + \dots$

```
// 求解第二类斯特林数第 k 列，求前 n 项，存入 ans
void get_S2_col(int n,int k,int *ans) {
    if (k>=n) {memset(ans,0,4*n);return;}
    static int A[N],B[N],C[N];
    memset(A,0,4*(n-k));
    for (int i=0;i<n-k;i++) A[i]=invfac[i+1];
    _ln(A,B,n-k);
    for (int i=0;i<n-k;i++) B[i]=1ll*B[i]*k%mod;
    _exp(B,C,n-k);
    memset(ans,0,4*n);
    int inv_k=invfac[k];
    for (int i=k;i<n;i++){
        ans[i]=1ll*C[i-k]*inv_k%mod*fac[i]%mod;
    }
}
```

有符号第一类斯特林数

$$s(n,k)=(-1)^{n-k}\begin{bmatrix}n\\k\end{bmatrix}$$

```
// 求解有符号第一类斯特林数第 n 行，存入 ans (长度为 n+1)
void get_signed_S1_row(int n,int *ans) {
    get_S1_row(n,ans); // 调用无符号版本
    for (int i=0;i<=n;i++) {
        if ((n-i)&1) ans[i]=dec(0,ans[i]); // 若 n-k 为奇数，取相反数
    }
}
```

贝尔数

贝尔数 B_n 表示将 n 个不同的元素划分为若干个非空集合的方案数。它与第二类斯特林数有着密不可分的联系：第 n 个贝尔数就是第二类斯特林数第 n 行的求和。

$$B_n=\sum_{k=0}^{n}\begin{Bmatrix}n\\k\end{Bmatrix}$$

$$F(x)=\sum_{n=0}^{\infty}B_n\frac{x^n}{n!}=e^{e^x-1}$$

```
// 求前 n 个贝尔数 (0 到 n-1)，存入 ans
void get_Bell(int n,int *ans) {
    static int A[N];
    memset(A,0,4 * n);
    for (int i=1;i<n;i++) A[i]=invfac[i];
    _exp(A,ans,n);
    for (int i=0;i<n;i++) ans[i]=1ll*ans[i]*fac[i]%mod;
}
```

斯特林反演

$$f_n=\sum_{k=0}^{n}\begin{Bmatrix}n\\k\end{Bmatrix}g_k\iff g_n=\sum_{k=0}^{n}(-1)^{n-k}\begin{bmatrix}n\\k\end{bmatrix}f_k$$

```
// 场景：已知 g 数组的前 n 项，利用斯特林反演求解单个特定的 f_n
int solve_stirling_inv_fn(int n,int *g) {
    static int S2[N];
    get_S2_row(n, S2);
    int fn=0;
    for (int k=0;k<=n; k++) {
        fn=add(fn,1ll*S2[k]*g[k]%mod);
    }
    return fn;
}

// 场景：已知 f 数组的前 n 项，利用斯特林反演求解单个特定的 g_n
int solve_stirling_inv_gn(int n,int *f) {
    static int S1[N];
    get_S1_row(n, S1);
    int gn = 0;
    for (int k=0;k<=n;k++) {
        int term=1ll*S1[k]*f[k]%mod;
        if ((n-k)&1) gn=dec(gn,term); // 奇数取负
        else gn=add(gn,term);             // 偶数取正
    }
    return gn;
}
```

FWT - OR (按位或卷积)

$C_k = \sum_{i|j=k} A_i B_j$
于按位或运算，若 $i \mid j = k$，那么 i 和 j 的二进制位必须是 k 的子集。因此，正变换本质上是求**高维前缀和**（子集和）。

```
// 按位或 (OR) 卷积
void FWT_or(int *a,int n,int op) {
    for (int k=2,m=1;k<=n;m<<=1,k<<=1) {
        for (int i=0;i<n;i+=k) {
            for (int j=i;j<i+m;j++) {
                if (op==1) a[j+m]=add(a[j+m],a[j]);
                else a[j+m]=dec(a[j+m],a[j]);
            }
        }
    }
}
```

FWT - AND (按位与卷积)

$C_k = \sum_{i\&j=k} A_i B_j$
对于按位与运算，若 $i\&j = k$，那么 i 和 j 的二进制位必须是 k 的超集。因此，正变换本质上是求**高维后缀和**（超集和）

```
// 按位与 (AND) 卷积
void FWT_and(int *a,int n,int op) {
    for (int k=2,m=1;k<=n;m<<=1,k<<=1) {
        for (int i=0;i<n;i+=k) {
            for (int j=i;j<i+m;j++) {
                if (op==1) a[j]=add(a[j],a[j+m]);
                else a[j]=dec(a[j],a[j+m]);
            }
        }
    }
}
```

FWT - XOR (按位异或卷积)

$C_k = \sum_{i \oplus j = k} A_i B_j$

构造变换矩阵 $\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$。正变换：$A'_0 = A_0 + A_1$，$A'_1 = A_0 - A_1$。因为矩阵行列式不为 1，逆变换时求解逆矩阵需要除以 2（即乘以 2 的逆元）：$A_0 = \frac{A'_0 + A'_1}{2}$，$A_1 = \frac{A'_0 - A'_1}{2}$。

```
// 按位异或 (XOR) 卷积
void FWT_xor(int *a,int n,int op) {
    static const int inv2=(mod+1)>>1;
    for (int k=2,m=1;k<=n;m<<=1,k<<=1) {
        for (int i=0;i<n;i+=k) {
            for (int j=i;j<i+m;j++) {
                int x=a[j],y=a[j+m];
                a[j]=add(x,y);
                a[j+m]=dec(x,y);
                if (op==-1) {
                    a[j]=1ll*a[j]*inv2%mod;
                    a[j+m]=1ll*a[j+m]*inv2%mod;
                }
            }
        }
    }
}
```

FWT - XNOR (按位同或卷积)

$C_k = \sum_{i \odot j = k} A_i B_j$

基于同或的真值表，构造出变换矩阵 $\begin{pmatrix} -1 & 1 \\ 1 & 1 \end{pmatrix}$。正变换：$A'_0 = -A_0 + A_1$，$A'_1 = A_0 + A_1$。逆变换同样具有对称性并需除以 2：$A_0 = \frac{A'_1 - A'_0}{2}$，$A_1 = \frac{A'_1 + A'_0}{2}$。

```
// 按位同或 (XNOR) 卷积
void FWT_xnor(int *a,int n,int op) {
    static const int inv2=(mod+1)>>1;
    for (int k=2,m=1;k<=n;m<<=1,k<<=1) {
        for (int i=0;i<n;i+=k) {
            for (int j=i;j<i+m;j++) {
                int x=a[j],y=a[j+m];
                if (op==1) {
                    a[j]=dec(y,x);       // -x + y
                    a[j+m]=add(x,y);   //  x + y
                } else {
                    a[j]=1ll*dec(y,x)*inv2%mod;
                    a[j+m]=1ll*add(x,y)*inv2%mod;
                }
            }
        }
    }
}
```

简单拉格朗日插值

```
// 场景：已知 n 个点 (1, y[1]), (2, y[2]), ..., (n, y[n])
int lagrange_continuous(int n,int *y,int k) {
    if (k>=1&&k<=n) return y[k];
    static int pre[N], suf[N];
    pre[0]=1;
    for (int i=1;i<=n;i++) pre[i]=1ll*pre[i-1]*dec(k,i)%mod;
    suf[n+1]=1;
    for (int i=n;i>=1;i--) suf[i]=1ll*suf[i+1]*dec(k,i)%mod;
    int ans=0;
    for (int i=1;i<=n;i++) {
        int num=1ll*pre[i-1]*suf[i+1]%mod;
        int den=1ll*invfac[i-1]*invfac[n-i]%mod;
        int term=1ll*y[i]*num%mod*den%mod;
        if ((n-i)&1)ans=dec(ans,term);
        else ans=add(ans,term);
    }
    return ans;
}
```

分圆多项式

```
// 求莫比乌斯函数单点值
int get_mu(int n) {
    int res=1;
    for (int i=2;i*i<=n;i++) {
        if (n%i==0) {
            res=-res;
            n/=i;
            if (n%i==0) return 0; // 包含平方因子
        }
    }
    if (n>1) res=-res;
    return res;
}
// 求第 n 个分圆多项式 Phi_n(x)，存入 ans// 返回值：该多项式的度数 (即 phi(n))int get_cyclotomic(int n,int *ans) {
    memset(ans,0,4*(n+1));
    ans[0]=1; // 初始为 1    int deg=0; // 当前多项式的度数
    for (int d=1;d<=n;d++) {
        if (n%d!=0) continue;
        int mu=get_mu(n/d);
        if (mu==0) continue;
        if (mu==1) {
            // 乘以 (x^d - 1)            // F_{new}(x) = F(x) * x^d - F(x)            for (int i=deg;i>=0;i--) {
                ans[i+d]=add(ans[i+d],ans[i]);
                ans[i]=dec(0,ans[i]); // 乘 -1            }
            deg+=d;
        } else if (mu==-1) {
            // 除以 (x^d - 1)            // F_{new}(x) * (x^d - 1) = F(x)            // 递推：F_{new}[i] = -F[i] + F
            for (int i=0;i<=deg-d;i++) {
                int val=dec(0,ans[i]);
                ans[i]=val;
                ans[i+d]=add(ans[i+d],val);
            }
            deg-=d;
            for (int i=deg+1;i<=deg+d;i++) ans[i]=0;
        }
    }
    return deg;
}
```

核心积性函数与数论基础

这一部分是整个数论大厦的基石，主要讨论整数域下的函数性质。

1. 基础定义

- **完全积性函数**：对于函数 f，满足 $f(ab) = f(a)f(b)$。
- **积性函数**：对于函数 f，仅在 $\gcd(a, b) = 1$ 时，满足 $f(ab) = f(a)f(b)$。

2. 常见积性函数图鉴

熟练掌握以下核心函数的定义与性质，是后续推导狄利克雷卷积与反演的基础：

函数名称	符号	定义与计算公式	备注
单位函数	$I(n)$	$I(n)=1$	积性函数
元函数	$\varepsilon(n)$	$\varepsilon(n)=[n=1]$	积性函数
恒等函数	$id(n)$	$id(n)=n$	积性函数
欧拉函数	$\varphi(n)$	$\varphi(n)=\sum_{i=1}^{n}[\gcd(i,n)=1]=n\prod(1-\frac{1}{p_i})$	p_i 为 n 唯一分解中的互异质数
莫比乌斯函数	$\mu(n)$	$n=1$ 时 $\mu(n)=1$；唯一分解的质数次数全为 1 时为 $(-1)^k$（k 为质数种类）；否则为 0	容斥原理在数论中的核心体现
约数个数	$d(n)$	表示 n 的约数个数	积性函数
除数函数	$\sigma_k(n)$	表示 n 的约数的 k 次方和	$\sigma(n)$ 特指约数和

埃拉托斯特尼筛法 (Eratosthenes)

```
int vis[N],pri[N],cnt;
void Eratosthenes(int n){
    vis[0]=vis[1]=0;cnt=0;
    for(int i=2;i<=n;i++)vis[i]=1;
    for (int i=2;i<=n;i++){
        if (vis[i]){
            pri[cnt++]=i;
            if (1ll*i*i>n)continue;
            for (int j=i*i;j<=n;j+=i){
                vis[j]=0;
            }
        }
    }
}
```

线性筛（欧拉筛）

```
void Euler(int n){
    cnt=0;
    for (int i=2;i<=n;i++)vis[i]=0;
    for (int i=2;i<=n;i++){
        if (!vis[i])pri[cnt++]=i;
        for (int j=1;j<=cnt;j++){
            if (1ll*i*pri[j]>n)break;
            vis[i*pri[j]]=1;
            if (i%pri[j]==0)break;
        }
    }
}
```

线性筛求欧拉函数 $\varphi(n)$

若 $\gcd(m,p_1)\neq 1$，则 $\varphi(n)=pri[j]\times\varphi(i)$；若 $\gcd(m,p_1)=1$，则 $\varphi(n)=(pri[j]-1)\times\varphi(i)$。

```
int phi[N];
void get_phi(int n){
    cnt=0;phi[1]=1;
    for (int i=2;i<=n;i++)vis[i]=0;
    for (int i=2;i<=n;i++){
        if (!vis[i])pri[++cnt]=i,phi[i]=i-1;
        for (int j=1;j<=cnt;j++){
            if (1ll*i*pri[j]>n)break;
            if (i%pri[j]==0){
                vis[i*pri[j]]=1;phi[i*pri[j]]=pri[j]*phi[i];
                break;
            }
            vis[i*pri[j]]=1;phi[i*pri[j]]=(pri[j]-1)*phi[i];
        }
    }
}
```

线性筛求莫比乌斯函数 $\mu(n)$

质数处 $\mu(i)=-1$，若遇到平方因子（即 $i \bmod pri[j]==0$），则 $\mu(i\times pri[j])=0$。

```
int mu[N];
void get_mu(int n){
    cnt=0;mu[1]=1;
    for (int i=2;i<=n;i++)vis[i]=0;
    for (int i=2;i<=n;i++){
        if (!vis[i])pri[++cnt]=i,mu[i]=-1;
        for (int j=1;j<=cnt;j++){
            if (1ll*i*pri[j]>n)break;
            if (i%pri[j]==0){
                vis[i*pri[j]]=1;mu[i*pri[j]]=0;
                break;
            }
            vis[i*pri[j]]=1;mu[i*pri[j]]=-mu[i];
        }
    }
}
```

线性筛求约数个数 $d(n)$

通过数组 num 维护最小质因子的个数，利用公式 $d(i\times pri[j])=d(i)\times(num[i\times pri[j]]+1)/num[i\times pri[j]]$ 进行转移。

```
int d[N],num[N];
void get_d(int n){
    cnt=0;d[1]=1;
    for (int i=2;i<=n;i++)vis[i]=0;
    for (int i=2;i<=n;i++){
        if (!vis[i])pri[++cnt]=i,d[i]=2,num[i]=1;
        for (int j=1;j<=cnt;j++){
            if (1ll*i*pri[j]>n)break;
            if (i%pri[j]==0){
                vis[i*pri[j]]=1;num[i*pri[j]]=num[i]+1;d[i*pri[j]]=d[i]*(num[i*pri[j]]+1)/num[i*pri[j]];
                break;
            }
            vis[i*pri[j]]=1;num[i*pri[j]]=1;d[i*pri[j]]=2*d[i];
        }
    }
}
```

线性筛求约数和 $\sigma(n)$

通过数组 $gsum$ 维护最小质因子的等比数列求和项 $g_i=\sum_{t=0}^{num_i}p_1^t$，以此来递推约数和 $dsum$。

```
int dsum[N],gsum[N];
void get_dsum(int n){
    cnt=0;dsum[1]=1;
    for (int i=2;i<=n;i++)vis[i]=0;
    for (int i=2;i<=n;i++){
        if (!vis[i])pri[++cnt]=i,dsum[i]=1+i,gsum[i]=1+i;
        for (int j=1;j<=cnt;j++){
            if (1ll*i*pri[j]>n)break;
            if (i%pri[j]==0){
                vis[i*pri[j]]=1;
                gsum[i*pri[j]]=pri[j]*gsum[i]+1;
                dsum[i*pri[j]]=dsum[i]*(gsum[i*pri[j]])/gsum[i];
                break;
            }
            vis[i*pri[j]]=1;
            gsum[i*pri[j]]=1+pri[j];
            dsum[i*pri[j]]=(1+pri[j])*dsum[i];
        }
    }
}
```

亚线性高级筛法

杜教筛 (时间复杂度 $O(n^{\frac{3}{4}})$预处理后可达 $O(n^{\frac{2}{3}})$):

用途：用于低于线性时间内求 $S(n)=\sum_{i=1}^{n}f(i)$。
核心构造：寻找辅助函数 g，使得 g 与 $f*g$ 的前缀和均可快速计算。
核心递推方程：

$$g(1)S(n)=\sum_{i=1}^{n}(f*g)(i)-\sum_{i=2}^{n}g(i)S(\lfloor\frac{n}{i}\rfloor)$$

以同时求 φ 和 μ 的前缀和为例

```cpp
#define N 5000050
#define ll long long
using std::unordered_map;
unordered_map<int,ll>mp1,mp2;
bool vis[N];
int prime[N],mu[N],phi[N],tot;
int smu[N];
ll sphi[N];
// 1. 线性预处理前 N 项的函数值及其前缀和
void pre_phi(){
    phi[1]=1;mu[1]=1;
    for (int i=2;i<N;i++){
        if (!vis[i])prime[++tot]=i,phi[i]=i-1,mu[i]=-1;
        for (int j=1;j<=tot&&i*prime[j]<N;j++){
            vis[i*prime[j]]=1;
            if (i%prime[j]==0){
                phi[i*prime[j]]=phi[i]*prime[j];
                mu[i*prime[j]]=0;
                break;
            }
            phi[i*prime[j]]=phi[i]*(prime[j]-1);
            mu[i*prime[j]]=-mu[i];
        }
    }
    for (int i=1;i<N;i++)sphi[i]=sphi[i-1]+phi[i],smu[i]=smu[i-1]+mu[i];
}
// 2. 杜教筛求 phi 的前缀和
ll get_S_phi(ll n){
    if (n<N)return sphi[n];
    if (mp1.count((int)n))return mp1[(int)n]; // 记忆化
    ll res=1ll*n*(1ll*n+1)/2; // 计算 (f*g) 的前缀和，此处为 id 的前缀和
    ll l=2,r;
    // 数论分块计算后半部分
    while (l<=n){
        r=n/(n/l);
        res-=1ll*(r-l+1)*get_S_phi(n/l);
        l=r+1;
    }
    return mp1[(int)n]=res;
}
// 3. 杜教筛求 mu 的前缀和
ll get_S_mu(ll n){
    if (n<N)return smu[n];
    if (mp2.count((int)n))return mp2[(int)n];
    ll res=1ll; // 计算 (f*g) 的前缀和，此处为 \epsilon 的前缀和，始终为 1
    ll l=2,r;
    while (l<=n){
        r=n/(n/l);
        res-=1ll*(r-l+1)*get_S_mu(n/l);
        l=r+1;
    }
```

```
    return mp2[(int)n]=res;
}
```

Min_25 筛：

适用条件：$f(p)$ 为关于 p 的可快速求值的完全积性函数之和，且 $f(p^c)$ 可快速求值。
核心步骤一 (求素数处贡献)：求 $F_{\text{prime}}(n)=\sum_{p\le n} f(p)$。利用滚动数组递推 $G_k(n)=G_{k-1}(n)-[p_k^2\le n]g(p_k)(G_{k-1}(\lfloor\frac{n}{p_k}\rfloor)-G_{k-1}(p_{k-1}))$，状态点仅存在于 $\lfloor\frac{n}{i}\rfloor$ 的位置。
$G_k(n)$ 表示埃筛第 k 轮筛后剩下的 g 的和。
核心步骤二 (合并计算 $F_k(n)$)：从大到小枚举 p 倒推计算，当 $p^2<n$ 时转移增加值不为零，可通过后缀和技巧进行深度优化。
利用 $F_k(n)$ 表示范围 $1\dots n$ 内，最小质因子 $>p_k$ 的数的函数值之和（最终答案为 $F_0(n)+f(1)$）。

$$F_k(n)=\sum_{k\le i,\,p_i^2\le n}\sum_{c\ge 1,\,p_i^{c+1}\le n}\left(f(p_i^{c+1})+f(p_i^c)F_{i+1}(\lfloor n/p_i^c\rfloor)\right)+F_{\text{prime}}(n)-F_{\text{prime}}(p_{k-1}) \tag{1}$$

求 $f(p)=p^2-p$ 的前缀和为例

```cpp
#define ll long long
#define N 1000500
#define mod 1000000007
bool vis[N];
ll prime[N],tot,cnt;
ll n,m,id1[N],id2[N],s1[N],s2[N],w[N],inv3,g1[N],g2[N];
// 1. 预处理 sqrt(n) 以内的质数，并维护质数一次项和二次项的前缀和
void pre_prime(int n){
    for (int i=2;i<=n;i++){
        if (!vis[i])prime[++tot]=i,s1[tot]=(s1[tot-1]+i)%mod,s2[tot]=(s2[tot-1]+1ll*i*i%mod)%mod;
        for (int j=1;j<=tot&&1ll*i*prime[j]<=n;j++){
            vis[i*prime[j]]=1;
            if (i%prime[j]==0)break;
        }
    }
}
// 2. 离散化所有形如 n/i 的状态
void init_g(){
    for (ll i=1,j;i<=n;i=j+1){
        j=n/(n/i);
        w[++cnt]=n/i;
        ll t=w[cnt]%mod;
        // 初始化 g(x) 为当所有数都是质数时的求和
        g1[cnt]=(1ll*t*(t+1)/2-1)%mod; // \sum_{i=2}^x i
        g2[cnt]=(1ll*t*(t+1)/2%mod*(2*t+1)%mod*inv3%mod+mod-1)%mod; // \sum_{i=2}^x i^2
        if (n/i<=m)id1[n/i]=cnt;
        else id2[j]=cnt;
    }
}
// 3. 类似埃氏筛递推，求质数部分的贡献 G_k(n)
void init_G(){
    for (int i=1;i<=tot;i++){
        for (int j=1;j<=cnt&&prime[i]*prime[i]<=w[j];j++){
            ll k=(w[j]/prime[i]<=m)?id1[w[j]/prime[i]]:id2[n/(w[j]/prime[i])];
            g1[j]=((g1[j]-prime[i]*(g1[k]-s1[i-1]+mod)%mod)+mod)%mod;
            g2[j]=((g2[j]-prime[i]*prime[i]%mod*(g2[k]-s2[i-1]+mod)%mod)+mod)%mod;
        }
    }
}
// 4. 递归求解 F_k(n)
ll F(int k,ll nn){
    if (prime[k]>=nn)return 0;
    ll x=(nn<=m)?id1[nn]:id2[n/nn];
    // 质数部分贡献，此处 f(p) = p^2 - p，所以对应 g2 - g1
    ll res=(-g1[x]+g2[x]+mod+s1[k]-s2[k]+mod)%mod;
    // 枚举 >p_k 的质因数及其幂次计算合数部分贡献
    for (int i=k+1;i<=tot&&prime[i]*prime[i]<=nn;i++){
        ll p=prime[i];
        for (int c=1;p<=nn;c++,p*=prime[i]){
            ll tmp=p%mod;
            // 积性函数性质：f(p^c) * f(...)，其中 f(p^c) 此题即为 tmp * (tmp - 1)
```

```
            res=(res+tmp*(tmp-1)%mod*(F(i,nn/p)+(c!=1)))%mod;
        }
    }
    return res%mod;
}
// 最终调用: ans = (F(0, n) + 1) % mod;
```

狄利克雷卷积

交换律：$f*g=g*f$
结合律：$(f*g)*h=f*(g*h)$（**最核心的推导武器**）
分配律：$f*(g+h)=f*g+f*h$
单位元：$f*\varepsilon=f$

卷积公式	展开形式	物理意义/用途
$\mu*1=\varepsilon$	$\sum_{d\|n}\mu(d)=[n=1]$	**莫比乌斯反演的核心灵魂**。常用于将 $[\gcd(i,j)=1]$ 拆解为 $\sum_{d\|\gcd(i,j)}\mu(d)$。
$\varphi*1=id$	$\sum_{d\|n}\varphi(d)=n$	欧拉函数的等价刻画。杜教筛求 φ 前缀和的构造基础。
$1*1=d$	$\sum_{d\|n}1=d(n)$	约数个数函数的生成方式。
$id*1=\sigma$	$\sum_{d\|n}d=\sigma(n)$	约数和函数的生成方式。

公式：$\varphi=\mu*id$
展开：$\varphi(n)=\sum_{d|n}\mu(d)\frac{n}{d}$
推导：因为 $\varphi*1=id$，两边同乘 μ，得到 $\varphi*1*\mu=id*\mu$。因为 $1*\mu=\varepsilon$，所以 $\varphi*\varepsilon=id*\mu$，即 $\varphi=\mu*id$

公式：$\mu*d=1$
推导：$d=1*1$，两边同乘 μ，得到 $\mu*d=\mu*1*1=\varepsilon*1=1$。
公式：$\mu*\sigma=id$
推导：$\sigma=id*1$，两边同乘 μ，得到 $\mu*\sigma=\mu*1*id=\varepsilon*id=id$。

公式：$\varphi*d=\sigma$
推导：$\varphi*d=\varphi*(1*1)=(\varphi*1)*1=id*1=\sigma$。这个式子在处理涉及约数个数与欧拉函数乘积的式子时有奇效。

公式：$id_k*1=\sigma_k$（定义）
公式：$\sigma_k*\mu=id_k$
展开：$\sum_{d|n}\sigma_k(d)\mu(\frac{n}{d})=n^k$

若 h 为完全积性函数，则 $(f\cdot h)*(g\cdot h)=(f*g)\cdot h$。（点乘代表对应项相乘，即 $f(x)h(x)$）。

$\sum_{i=1}^{n}i\cdot\varphi(i)$
$f=id\cdot\varphi$
$\varphi*1=id$

$$(\varphi \cdot id) * (1 \cdot id) = (id \cdot id)$$

$f * id = id_2$
所以我们选择 $g = id$（前缀和为 $\frac{n(n+1)}{2}$），此时 $f * g = id_2$（前缀和为 $\frac{n(n+1)(2n+1)}{6}$）

$f(n) = n^2 \cdot \mu(n)$
已知 $\mu * 1 = \varepsilon$，两边点乘 id_2，得到 $(\mu \cdot id_2) * (1 \cdot id_2) = (\varepsilon \cdot id_2) = \varepsilon$。 所以选择 $g = id_2$，$f * g = \varepsilon$，即可直接套用杜教筛模板

莫比乌斯反演 (Möbius Inversion)

普通形式：$f(n) = \sum_{d|n} g(d) \iff g(n) = \sum_{d|n} \mu(\frac{n}{d}) f(d)$。等价于狄利克雷卷积形式 $f = 1 * g \iff g = \mu * f$。
常见应用：利用 $\varepsilon = 1 * \mu$，将 $[\gcd(i,j) = 1]$ 巧妙拆解为 $\sum_{d|\gcd(i,j)} \mu(d)$，从而改变求和顺序。
拓展形式：
倍数反演：$f(n) = \sum_{n|d} g(d) \iff g(n) = \sum_{n|d} \mu(\frac{d}{n}) f(d)$。
乘积反演：$f(n) = \prod_{d|n} g(d) \iff g(n) = \prod_{d|n} f(d)^{\mu(\frac{n}{d})}$。
Dirichlet 逆：$f = \alpha * g \iff g = \alpha^{-1} * f$，其中 α^{-1} 为 α 的 Dirichlet 逆。

数论分块 (Number Theory Block)

用于在 $O(\sqrt{n})$ 的时间复杂度内计算形如 $\sum_{i=1}^{n} f(i) g(\lfloor \frac{n}{i} \rfloor)$ 的式子。
一维分块：区间右端点公式为 $r = \lfloor \frac{n}{\lfloor \frac{n}{l} \rfloor} \rfloor$。
向上取整分块：计算 $\sum_{i=1}^{n} f(i) g(\lceil \frac{n}{i} \rceil)$ 时，利用恒等式 $\lceil \frac{n}{i} \rceil = \lfloor \frac{n-1}{i} \rfloor + 1$ 转化为向下取整模型处理。
多维分块：计算包含多个向下取整项（如 $\lfloor \frac{n_1}{i} \rfloor, \lfloor \frac{n_2}{i} \rfloor$ 等）的乘积时，右端点只需取所有维度右端点的最小值，即 $r = \min_{i=1}^{m} \lfloor \frac{n_i}{\lfloor \frac{n_i}{l} \rfloor} \rfloor$。
任意指数分块：处理形如 $\sum_{i=1}^{\lfloor n^{\frac{a}{b}} \rfloor} f(i) g(\lfloor \frac{n^a}{i^b} \rfloor)$ 时，区间边界公式严格推导为 $\lfloor \frac{n^{\frac{a}{b}}}{(d+1)^{\frac{1}{b}}} \rfloor + 1 \le i \le \lfloor \frac{n^{\frac{a}{b}}}{d^{\frac{1}{b}}} \rfloor$，整体复杂度降为 $O(n^{\frac{a}{b+1}})$。

Dirichlet 前缀和

高维前缀和思想：将每一个素数视为一个独立维度，从小到大遍历所有素数 p，并将 n 处的函数值累加到 $n \times p$ 处。
复杂度与应用：这与 Eratosthenes 筛法的遍历顺序完全一致，可在 $O(n \log \log n)$ 时间内极其高效地计算出长度为 n 的数列的 Dirichlet 前缀和或进行逐维差分。

```
int f[N],g[N];
void di_presum(int n){
    for (int i=0;i<=n;i++)vis[i]=0,f[i]=g[i];
    for (int i=2;i<=n;i++){
        if (vis[i])continue;
        // 从小到大枚举素数，正向累加计算前缀和
        for (int j=1,ij=i;i*j<=n;j++,ij+=j){
            vis[ij]=1;
            f[ij]+=g[j];
        }
    }
}
void di_diff(int n){
    for (int i=0;i<=n;i++)vis[i]=0,g[i]=f[i];
    for (int i=2;i<=n;i++){
        if (vis[i])continue;
        // 倒序枚举倍数，利用逐维差分还原数组
        for (int j=n/i,ij=i*j;j>=1;j--,ij-=j){
            vis[ij]=1;
            g[ij]-=f[j];
        }
    }
}
```

类欧几里得算法 (Euclidean-like Algorithm)

基础函数 f：$f(a,b,c,n)=\sum_{i=0}^{n}\lfloor\frac{ai+b}{c}\rfloor$。推导核心是先提取 $\lfloor a/c\rfloor$ 和 $\lfloor b/c\rfloor$，并在 $a<c, b<c$ 时通过交换求和次序，转化为关于 $m=\lfloor\frac{an+b}{c}\rfloor$ 的递归式 $mn-f(c,c-b-1,a,m-1)$。
平方函数 g：$g(a,b,c,n)=\sum_{i=0}^{n}\lfloor\frac{ai+b}{c}\rfloor^2$。
乘积函数 h：$h(a,b,c,n)=\sum_{i=0}^{n}i\lfloor\frac{ai+b}{c}\rfloor$。
相互依赖：g 和 h 的递归降阶过程并非孤立，而是会相互调用，并依赖于 f 的结果，最终在 $O(\log\max(a,c))$ 时间内完成收敛。

函数 $f(a,b,c,n)$ 的推导

定义为：$f(a,b,c,n)=\sum_{i=0}^{n}\lfloor\frac{ai+b}{c}\rfloor$。
第一步：提取整数部分

$$
\begin{aligned}
f(a,b,c,n) &= \sum_{i=0}^{n}\left\lfloor\frac{ai+b}{c}\right\rfloor && (2)\\
&= \sum_{i=0}^{n}\left\lfloor\frac{(\lfloor a/c\rfloor\cdot c+(a \bmod c))i+\lfloor b/c\rfloor\cdot c+(b \bmod c)}{c}\right\rfloor && (3)\\
&= \sum_{i=0}^{n}\left\lfloor\frac{(a \bmod c)i+(b \bmod c)}{c}\right\rfloor+\lfloor a/c\rfloor\, i+\lfloor b/c\rfloor && (4)\\
&= \frac{n(n+1)}{2}\lfloor a/c\rfloor+(n+1)\lfloor b/c\rfloor+f(a \bmod c,\, b \bmod c,\, c,\, n) && (5)
\end{aligned}
$$

第二步：坐标轴变换（当 $a<c, b<c$ 时） 令 $m=\lfloor\frac{an+b}{c}\rfloor$，且利用 $\lfloor\frac{ai+b}{c}\rfloor=\lceil\frac{ai+b+1}{c}\rceil-1$。

$$
\begin{aligned}
f(a,b,c,n) &= \sum_{i=0}^{n} \left\lfloor \frac{ai+b}{c} \right\rfloor && (6)\\
&= \sum_{i=0}^{n} \sum_{j=0}^{m-1} [\, j < \left\lfloor \frac{ai+b}{c} \right\rfloor \,] && (7)\\
&= \sum_{i=0}^{n} \sum_{j=0}^{m-1} [\, j < \left\lceil \frac{ai+b+1}{c} \right\rceil - 1 \,] && (8)\\
&= \sum_{i=0}^{n} \sum_{j=0}^{m-1} [\, j+1 < \left\lceil \frac{ai+b+1}{c} \right\rceil \,] && (9)\\
&= \sum_{i=0}^{n} \sum_{j=0}^{m-1} [\, j+1 < \frac{ai+b+1}{c} \,] && (10)\\
&= \sum_{i=0}^{n} \sum_{j=0}^{m-1} [\, jc+c < ai+b+1 \,] && (11)\\
&= \sum_{j=0}^{m-1} \sum_{i=0}^{n} [\, i > \frac{cj+c-b-1}{a} \,] && (12)\\
&= \sum_{j=0}^{m-1} \sum_{i=0}^{n} [\, i > \left\lfloor \frac{cj+c-b-1}{a} \right\rfloor \,] && (13)\\
&= \sum_{j=0}^{m-1} \left(n - \left\lfloor \frac{cj+c-b-1}{a} \right\rfloor \right) && (14)\\
&= mn - f(c,\, c-b-1,\, a,\, m-1) && (15)
\end{aligned}
$$

函数 $g(a,b,c,n)$ 的推导

定义为：$g(a,b,c,n) = \sum_{i=0}^{n} \lfloor \frac{ai+b}{c} \rfloor^2$。

第一步：提取整数部分

$$
\begin{aligned}
g(a,b,c,n) &= \sum_{i=0}^{n} \left\lfloor \frac{ai+b}{c} \right\rfloor^2 && (16)\\
&= \sum_{i=0}^{n} \left\lfloor \frac{(\lfloor a/c \rfloor \cdot c + (a \bmod c))i + \lfloor b/c \rfloor \cdot c + (b \bmod c)}{c} \right\rfloor^2 && (17)\\
&= \sum_{i=0}^{n} \left(\left\lfloor \frac{(a \bmod c)i + (b \bmod c)}{c} \right\rfloor + \lfloor a/c \rfloor\, i + \lfloor b/c \rfloor \right)^2 && (18)\\
&= \sum_{i=0}^{n} \left\lfloor \frac{(a \bmod c)i + (b \bmod c)}{c} \right\rfloor^2 + \sum_{i=0}^{n} (\lfloor a/c \rfloor\, i)^2 + \sum_{i=0}^{n} \lfloor b/c \rfloor^2 && (19)\\
&\quad + 2\lfloor a/c \rfloor \sum_{i=0}^{n} i \left\lfloor \frac{(a \bmod c)i + (b \bmod c)}{c} \right\rfloor + 2\lfloor b/c \rfloor \sum_{i=0}^{n} \left\lfloor \frac{(a \bmod c)i + (b \bmod c)}{c} \right\rfloor && (20)\\
&\quad + 2\lfloor a/c \rfloor \lfloor b/c \rfloor \sum_{i=0}^{n} i && (21)\\
&= g(a \bmod c,\, b \bmod c,\, c,\, n) + \frac{n(n+1)(2n+1)}{6} \lfloor a/c \rfloor^2 + (n+1) \lfloor b/c \rfloor^2 && (22)\\
&\quad + 2\lfloor a/c \rfloor\, h(a \bmod c,\, b \bmod c,\, c,\, n) + 2\lfloor b/c \rfloor\, f(a \bmod c,\, b \bmod c,\, c,\, n) + n(n+1) \lfloor a/c \rfloor \lfloor b/c \rfloor && (23)
\end{aligned}
$$

第二步：坐标轴变换（当 $a < c, b < c$ 且 $m = \lfloor \frac{an+b}{c} \rfloor$ 时）

$$
\begin{aligned}
g(a,b,c,n) &= \sum_{i=0}^{n} \left\lfloor \frac{ai+b}{c} \right\rfloor^2 && (24)\\
&= \sum_{i=0}^{n} \sum_{j=0}^{m-1} (2j+1) \left[j < \left\lfloor \frac{ai+b}{c} \right\rfloor \right] && (25)\\
&= \sum_{j=0}^{m-1} (2j+1) \sum_{i=0}^{n} \left[i > \left\lfloor \frac{cj+c-b-1}{a} \right\rfloor \right] && (26)\\
&= \sum_{j=0}^{m-1} (2j+1) \left(n - \left\lfloor \frac{cj+c-b-1}{a} \right\rfloor \right) && (27)\\
&= m^2 n - 2\, h(c,\, c-b-1,\, a,\, m-1) - f(c,\, c-b-1,\, a,\, m-1) && (28)
\end{aligned}
$$

函数 $h(a,b,c,n)$ 的推导

定义为：$h(a,b,c,n) = \sum_{i=0}^{n} i \lfloor \frac{ai+b}{c} \rfloor$。

第一步：提取整数部分

$$
\begin{aligned}
h(a,b,c,n) &= \sum_{i=0}^{n} \left\lfloor \frac{ai+b}{c} \right\rfloor i && (29)\\
&= \sum_{i=0}^{n} \left\lfloor \frac{(\lfloor a/c \rfloor \cdot c + (a \bmod c))\, i + \lfloor b/c \rfloor \cdot c + (b \bmod c)}{c} \right\rfloor i && (30)\\
&= \sum_{i=0}^{n} \left\lfloor \frac{(a \bmod c)i + (b \bmod c)}{c} \right\rfloor i + \lfloor a/c \rfloor\, i^2 + \lfloor b/c \rfloor\, i && (31)\\
&= h(a \bmod c,\, b \bmod c,\, c,\, n) + \frac{n(n+1)(2n+1)}{6} \lfloor a/c \rfloor + \frac{n(n+1)}{2} \lfloor b/c \rfloor && (32)
\end{aligned}
$$

第二步：坐标轴变换（当 $a<c, b<c$ 且 $m=\lfloor \frac{an+b}{c} \rfloor$ 时）

$$
\begin{aligned}
h(a,b,c,n) &= \sum_{i=0}^{n} \left\lfloor \frac{ai+b}{c} \right\rfloor i && (33)\\
&= \sum_{i=0}^{n} i \sum_{j=0}^{m-1} \left[j < \left\lfloor \frac{ai+b}{c} \right\rfloor \right] && (34)\\
&= \sum_{j=0}^{m-1} \sum_{i=0}^{n} i \left[i > \left\lfloor \frac{cj+c-b-1}{a} \right\rfloor \right] && (35)\\
&= \sum_{j=0}^{m-1} \sum_{i=\left\lfloor \frac{cj+c-b-1}{a} \right\rfloor+1}^{n} i && (36)\\
&= \sum_{j=0}^{m-1} \frac{\left(n+\left\lfloor \frac{cj+c-b-1}{a} \right\rfloor+1\right)\left(n-\left\lfloor \frac{cj+c-b-1}{a} \right\rfloor\right)}{2} && (37)\\
&= \frac{1}{2}\sum_{j=0}^{m-1} \left((n+1)n - \left\lfloor \frac{cj+c-b-1}{a} \right\rfloor - \left\lfloor \frac{cj+c-b-1}{a} \right\rfloor^2 \right) && (38)\\
&= \frac{1}{2}(n+1)nm - \frac{1}{2}f(c,\, c-b-1,\, a,\, m-1) - \frac{1}{2}g(c,\, c-b-1,\, a,\, m-1) && (39)
\end{aligned}
$$

```
// 核心递归函数
node so(ll a,ll b,ll c,ll n){
        node res;res.f=res.g=res.h=0; // 预处理自然数一次方和、二次方和
        ll n1=n*(n+1)%mod*inv2%mod;
        ll n2=n*(n+1)%mod*(2*n+1)%mod*inv6%mod; // 分支一：提取整数部分
        if (a>=c||b>=c){
                node tmp=so(a%c,b%c,c,n);
                res.f=(n1*(a/c)%mod+(n+1)*(b/c)%mod+tmp.f)%mod;
                res.g=(tmp.g+n2*(a/c)%mod*(a/c)%mod+(n+1)*(b/c)%mod*(b/c)%mod +2*(a/c)*tmp.h%mod+2*(b/c)*tmp.f%m
                res.h=(tmp.h+n2*(a/c)%mod+n1*(b/c)%mod)%mod;
        } // 分支二：坐标轴变换降阶
        else{
                ll m=(a*n+b)/c;
                if (!m)return res; // 递归终点
                node tmp=so(c,c-b-1,a,m-1);
                res.f=(n*m%mod-tmp.f)%mod;
                res.g=(m*m%mod*n%mod-2*tmp.h-tmp.f)%mod;
                res.h=(n1*m%mod-tmp.f*inv2%mod-tmp.g*inv2%mod)%mod;
        } // 统一处理负数取模问题
        res.f=(res.f%mod+mod)%mod;
        res.g=(res.g%mod+mod)%mod;
        res.h=(res.h%mod+mod)%mod;
        return res;
}
```

最大公因数、逆元与线性方程

```
ll power(ll a,ll b,ll p){
    ll res=1;
    while (b){
        if (b&1)res=res*a%p;
        b>>=1;
        a=a*a%p;
    }
    return res;
}

ll gcd(ll a,ll b){
    return b?gcd(b,a%b):a;
}

ll exgcd(ll a,ll b,ll&x,ll&y){
    if (!b)return x=1,y=0,a;
    ll d=exgcd(b,a%b,y,x);
    y-=a/b*x;
    return d;
}

ll inv(ll a,ll p){
    ll x,y;
    ll d=exgcd(a,p,x,y);
    if (d!=1)return -1;
    return (x%p+p)%p;
}
```

中国剩余定理体系 (CRT & exCRT)

基础 CRT：用于合并模数 $n_1, n_2, \ldots, n_k$ 两两互质的线性同余方程组。核心在于构造 $m_i = \frac{n}{n_i}$ 及其在模 n_i 下的逆元 m_i^{-1}，答案即为 $x = \sum_{i=1}^{k} a_i m_i m_i^{-1}$。
扩展 CRT (exCRT)：用于处理模数不互质的方程组。将两个方程 $x \equiv a_1 \pmod{m_1}$ 和 $x \equiv a_2 \pmod{m_2}$ 化为 $m_1 p + m_2 q = a_2 - a_1$。若 $\gcd(m_1, m_2) \nmid (a_2 - a_1)$ 则无解；否则通过 exgcd 解出 p 和 q，将其合并为新的同余方程 $x \equiv b \pmod{\operatorname{lcm}(m_1, m_2)}$，从而两两缩点。

```
ll a[N],r[N];
ll crt(int k){
    ll n=1,ans=0;
    for (int i=1;i<=k;i++)n*=r[i];
    for (int i=1;i<=k;i++){
        ll m=n/r[i];
        ans=(ans+a[i]*m%n*inv(m,r[i])%n)%n;
    }
    return (ans%n+n)%n;
}

ll excrt(int k){
    ll M=r[1],ans=a[1];
    for (int i=2;i<=k;i++){
        ll x,y,A=M,B=r[i],C=(a[i]-ans%B+B)%B;
        ll d=exgcd(A,B,x,y);
        if (C%d!=0)return -1;
        x=x*(C/d)%(B/d);
        ans+=x*M;
        M*=B/d;
        ans=(ans%M+M)%M;
    }
    return (ans%M+M)%M;
}
```

离散对数与大步小步算法 (BSGS & exBSGS)

离散对数定义：在模 m 原根 g 存在的前提下，满足 $g^k \equiv a \pmod m$ 的唯一整数 k ($0 \le k < \varphi(m)$) 即为离散对数 $\text{ind}_g a$。其性质类似于常规对数，可将模意义下的乘法转化为加法。
BSGS 算法：求解 $a^x \equiv b \pmod p$ (其中 $\gcd(a,p)=1$)。令 $x = A\lceil\sqrt{p}\rceil - B$，原方程化为 $a^{A\lceil\sqrt{p}\rceil} \equiv ba^B \pmod p$。预处理右侧存入 Hash 表（小步），再枚举左侧（大步）进行匹配，复杂度 $O(\sqrt{p})$。
exBSGS 算法：当 $\gcd(a,p) > 1$ 时，无法直接求逆元。不断提取公因数 $d = \gcd(a,p)$，将方程约分为 $\frac{a}{d}a^{x-1} \equiv \frac{b}{d} \pmod{\frac{p}{d}}$。迭代此过程直至 $\gcd(a, p_{new}) = 1$，即可转回普通 BSGS 求解。

```
ll BSGS(ll a,ll b,ll p){
    mp.clear();
    if (b==1)return 0;
    a%=p;b%=p;
    ll m=ceil(sqrt(p)),res=1;
    for (int i=0;i<m;i++){
        if (!mp.count(res)){
            mp[res]=i;
        }
        res=(res*a)%p;
    }
    ll A=inv(res,p);
    res=b;
    for (int i=0;i<m;i++){
        if (mp.count(res)){
            return i*m+mp[res];
        }
        res=res*A%p;
    }
    return -1;
}

ll exBSGS(ll a,ll b,ll m){
    a%=m;b%=m;
    if (b==1||m==1)return 0;
    ll cnt=0,d=1,ad=1;
    while ((d=gcd(a,m))^1){
        if (b%d)return -1;
        cnt++;b/=d;m/=d;
        ad=(ad*a/d)%m;
        if (ad==b)return cnt;
    }
    ll inv_ad=inv(ad,m);
    ll ans=BSGS(a,b*inv_ad,m);
    if (ans==-1)return -1;
    return ans+cnt;
}
```

升幂引理 (Lifting The Exponent Lemma, LTE)

升幂引理用于快速计算 $x^n \pm y^n$ 中含有某个素因子 p 的幂次 $\nu_p(x^n \pm y^n)$，其中 $\nu_p(A)$ 表示满足 $p^k \mid A$ 的最大整数 k（即 $p^{\nu_p(A)} \mid\mid A$）。

第一部分：当 $\gcd(n,p)=1$ 时

若 $p \mid x-y$，则 $\nu_p(x^n-y^n)=\nu_p(x-y)$。

若 $p \mid x+y$ 且 n 为奇数，则 $\nu_p(x^n+y^n)=\nu_p(x+y)$。

第二部分：当 p 为奇素数时

若 $p \mid x-y$，则 $\nu_p(x^n-y^n)=\nu_p(x-y)+\nu_p(n)$。

若 $p \mid x+y$ 且 n 为奇数，则 $\nu_p(x^n+y^n)=\nu_p(x+y)+\nu_p(n)$。

第三部分：当 $p=2$ 且 $2 \mid x-y$ 时

对奇数 n，$\nu_2(x^n-y^n)=\nu_2(x-y)$。

对偶数 n，$\nu_2(x^n-y^n)=\nu_2(x-y)+\nu_2(x+y)+\nu_2(n)-1$。
（特例）若 $4\mid x-y$，则由于 $\nu_2(x+y)=1$，有 $\nu_2(x^n-y^n)=\nu_2(x-y)+\nu_2(n)$。

Wilson 定理

对自然数 $n>1$，当且仅当 n 是素数时，$(n-1)!\equiv -1 \pmod n$。

阶乘余数 (计算 $(n!)_p \bmod p$)

将 $n!$ 中 p 的倍数提取出来，剩余部分按 p 的周期分组，利用 Wilson 定理的推论可化简为 $(\pm 1)^{\lfloor \frac{n}{p} \rfloor}\cdot (n \bmod p)!\cdot (\lfloor \frac{n}{p} \rfloor !)_p \pmod p$。

Legendre 公式

计算 $n!$ 中质因子 p 的次数公式为 $\nu_p(n!)=\sum_{i=1}^{\infty}\lfloor \frac{n}{p^i} \rfloor=\frac{n-S_p(n)}{p-1}$，其中 $S_p(n)$ 为 p 进制下 n 的各位数字之和。

Kummer 定理

计算组合数 $\binom{m}{n}$ 中质因子 p 的次数公式为 $\nu_p(\binom{m}{n})=\frac{S_p(n)+S_p(m-n)-S_p(m)}{p-1}$。

Lucas 定理与 exLucas

Lucas 定理：对于素数 p，求大组合数取模极其高效：$\binom{n}{k}\equiv \binom{\lfloor \frac{n}{p} \rfloor}{\lfloor \frac{k}{p} \rfloor}\binom{n \bmod p}{k \bmod p} \pmod p$。

exLucas 算法 (一般模数)：
1. 当模数 $m=p_1^{\alpha_1}p_2^{\alpha_2}\dots p_s^{\alpha_s}$ 不是素数时，拆分为多个模 $p_i^{\alpha_i}$ 的方程组。
2. 对于素数幂 p^α 的情况，利用公式 $\binom{n}{k}=p^{\nu_p(n!)-\nu_p(k!)-\nu_p((n-k)!)}\frac{(n!)_p}{(k!)_p((n-k)!)_p}$ 提取出所有的 p 因子，分别计算阶乘剩余部分。
3. 最后使用中国剩余定理 (CRT) 将所有结果合并。

```
ll C(ll n,ll m,ll p){
    if (m<0||m>n)return 0;
    return fac[n]*inv[m]%p*inv[n-m]%p;
}

ll Lucas(ll n,ll m,ll p){
    if (!m)return 1;
    return C(n%p,m%p,p)*Lucas(n/p,m/p,p)%p;
}
```

```cpp
// 计算阶乘除去 p 因子后的值取模 p^k
ll fac_p(ll n,ll p,ll pk){
    if (!n)return 1;
    ll res=1;
    for (ll i=1;i<=pk;i++){
        if (i%p)res=res*i%pk;
    }
    res=power(res,n/pk,pk);
    for (ll i=1;i<=n%pk;i++){
        if (i%p)res=res*i%pk;
    }
    return res*fac_p(n/p,p,pk)%pk;
}
// 计算 C(n, m) % p^k
ll C_pk(ll n,ll m,ll p,ll pk){
    ll cnt=0;
    for (ll i=n;i;i/=p)cnt+=i/p;
    for (ll i=m;i;i/=p)cnt-=i/p;
    for (ll i=n-m;i;i/=p)cnt-=i/p;
    ll a=fac_p(n,p,pk),b=fac_p(m,p,pk),c=fac_p(n-m,p,pk);
    return a*inv(b,pk)%pk*inv(c,pk)%pk*power(p,cnt,pk)%pk;
}
// exLucas 主函数，利用 CRT 合并
ll exLucas(ll n,ll m,ll mod){
    ll res=0,tmp=mod;
    for (ll i=2;i*i<=tmp;i++){
        if (tmp%i==0){
            ll pk=1;
            while (tmp%i==0)pk*=i,tmp/=i;
            ll cur=C_pk(n,m,i,pk);
            res=(res+cur*(mod/pk)%mod*inv(mod/pk,pk)%mod)%mod;
        }
    }
    if (tmp>1){
        ll cur=C_pk(n,m,tmp,tmp);
        res=(res+cur*(mod/tmp)%mod*inv(mod/tmp,tmp)%mod)%mod;
    }
    return res;
}
```

二次剩余与 Cipolla 算法

Euler 判别法：对奇素数 p 和满足 $\gcd(a,p)=1$ 的 a，若 a 为二次剩余，则 $a^{\frac{p-1}{2}}\equiv 1\pmod p$；若为二次非剩余，则 $a^{\frac{p-1}{2}}\equiv -1\pmod p$。
Legendre 符号：$\left(\frac{a}{p}\right)$ 的取值完全对应 Euler 判别法的结果，且具备完全积性：$\left(\frac{a_1a_2}{p}\right)=\left(\frac{a_1}{p}\right)\left(\frac{a_2}{p}\right)$。
Cipolla 算法：用于求解 $x^2\equiv a\pmod p$。

1. 不断随机寻找一个 r，使得 r^2-a 为模 p 的二次非剩余（期望两次即可找到）。
2. 定义虚数单位 $w^2\equiv r^2-a\pmod p$，并在复数域下进行运算。
3. 最终解即为 $\pm(r+w)^{\frac{p+1}{2}}$。

```
ll w,p;
struct comp{
    ll x,y;
    comp(ll _x=0,ll _y=0):x(_x),y(_y){}
};
comp operator*(comp a,comp b){
    return comp((a.x*b.x%p+a.y*b.y%p*w%p)%p,(a.x*b.y%p+a.y*b.x%p)%p);
}
comp power(comp a,ll b){
    comp res(1,0);
    while (b){
        if (b&1)res=res*a;
        b>>=1;
        a=a*a;
    }
    return res;
}
ll power(ll a,ll b){
    ll res=1;
    while (b){
        if (b&1)res=res*a%p;
        b>>=1;
        a=a*a%p;
    }
    return res;
}
ll Cipolla(ll a){
    if (a==0)return 0;
    if (power(a,(p-1)/2)!=1)return -1;
    ll r;
    while (1){
        r=rand()%p;
        w=(r*r%p-a+p)%p;
        if (power(w,(p-1)/2)==p-1)break;
    }
    comp res=power(comp(r,1),(p+1)/2);
    return res.x;
}
```

阶、原根与 Carmichael 函数

阶 (Order)：满足 $a^n \equiv 1 \pmod m$ 的最小正整数 n 称为 $\delta_m(a)$。同余式 $a^n \equiv 1 \pmod m$ 成立当且仅当 $\delta_m(a) \mid n$。
原根存在性：模 m 的原根存在，当且仅当 $m = 1, 2, 4, p^e, 2p^e$（p 为奇素数）。
原根判定定理：g 为模 m 的原根，当且仅当对于 $\varphi(m)$ 的每个素因子 q，均有 $g^{\frac{\varphi(m)}{q}} \not\equiv 1 \pmod m$。
原根个数：若原根存在，其个数为 $\varphi(\varphi(m))$。
Carmichael 函数 $\lambda(m)$：使 $a^n \equiv 1 \pmod m$ 对所有与 m 互素的 a 都成立的最小正整数。对于 $m = p^e$ (奇素数)，$\lambda(m) = p^{e-1}(p-1)$；对于 $m = 2^e (e \geq 3)$，$\lambda(m) = \frac{1}{2}\varphi(m)$。

```
ll prm[N],cnt;
void get_prime_factors(ll x){
    cnt=0;
    for (ll i=2;i*i<=x;i++){
        if (x%i==0){
            prm[++cnt]=i;
            while (x%i==0)x/=i;
        }
    }
    if (x>1)prm[++cnt]=x;
}
ll get_g(ll m){
    ll phi=m-1; // 仅以 m 为素数为例，若 m 为素数幂需改为求欧拉函数
    get_prime_factors(phi);
    for (ll g=2;g<=m-1;g++){
        bool flag=1;
        for (int i=1;i<=cnt;i++){
            if (power(g,phi/prm[i])==1){
                flag=0;
                break;
            }
        }
        if (flag)return g;
    }
    return -1;
}
```

高次同余与模意义下开方

Hensel 引理：用于将素数幂模 p^e 的同余方程降阶为素数模 p 的同余方程。通过分析 $f'(x_0) \pmod p$ 的值，来确定是否可以通过 $x = x_0 + p^{e-1}t$ 构造出解。

高次剩余存在性：设 g 是模 m 的原根，$d = \gcd(k, \varphi(m))$，$d' = \frac{\varphi(m)}{d}$。$a$ 为模 m 的 k 次剩余，当且仅当 $a^{d'} \equiv 1 \pmod m$。

改良 Tonelli-Shanks 算法：求解 $x^k \equiv a \pmod m$ 的核心机制。通过求原根，利用离散对数性质将指数方程转化为线性同余方程 $k \cdot \operatorname{ind}_g x \equiv \operatorname{ind}_g a \pmod{\varphi(m)}$，随后利用 BSGS 与 exgcd 配合求解。

普通 Nim 博弈

有 n 堆石子，两人轮流取，每次可从某一堆中取任意数量（至少 1 个），拿走最后一个石子者获胜。

先手必胜 $\iff a_1 \oplus a_2 \oplus \cdots \oplus a_n \neq 0$

反 Nim 博弈

规则同 Nim，但**拿走最后一个石子的人失败**（无路可走者胜）。

先手必胜 $\iff$

1. 所有堆的石子数均为 1，且异或和等于 0（即堆数为偶数）；
2. 存在至少一堆石子数大于 1，且异或和不等于 0。

阶梯 Nim

在 $1 \dots n$ 阶梯上各有若干石子。每次可以将第 i 阶上的任意数量石子移到第 $i-1$ 阶（第 0 阶为地面，放入地面的石子不可再动）。无法移动者输。
等价于**奇数阶梯**上的普通 Nim 博弈（偶数阶上的移动总是可以被对手在下一轮移到奇数阶复刻）。先手必胜 $\iff$ $\bigoplus_{i\text{为奇数}} a_i \neq 0$。

Nim-k

有 n 堆石子，每次最多可以从 k 堆中同时取任意数量的石子。无法移动者输。
将所有堆的石子数二进制展开，对每个二进制位求和。若存在某一位的和 $\pmod{(k+1)} \neq 0$，则先手必胜；反之先手必败。

SG 函数

用于复杂或状态转移有限的自定义 Nim 变种打表/搜值。
$SG(x) = \operatorname{mex}(\{SG(y) \mid x \to y\})$。
先手必胜 $\iff$ $SG(G_1) \oplus SG(G_2) \oplus \cdots \oplus SG(G_n) \neq 0$

巴什博弈 (Bash Game)

一堆共 n 个石子，两人轮流取，每次最多取 m 个，最少取 1 个。取走最后一个石子者获胜。
若 $n \pmod{(m+1)} \neq 0$，则先手必胜。

威佐夫博弈 (Wythoff Game)

两堆石子分别为 a 和 b，每次可以从一堆中取任意多，或者从两堆中取相同数量。最后取完者获胜。
建立在 **Beatty 定理** 基础上。必败态（奇异局势）满足 $a = \lfloor k \times \frac{1+\sqrt{5}}{2} \rfloor, b = a + k$（假设 $a \le b$，$k = 0, 1, 2 \dots$）。

斐波那契博弈 (Fibonacci Game)

一堆共 n 个石子，两人轮流取。先手第一次可以取任意多个，但不能全取完（至少取 1 个，最多取 $n-1$ 个）。之后的每一步，取的石子数不能超过上一次取的石子数的 **2 倍**。最后取完者获胜。
基于 **Zeckendorf 定理**（任何正整数都可以唯一表示为若干个不连续的斐波那契数之和）。**当且仅当 n 是斐波那契数时，先手必败。**

欧几里得博弈 (Euclid Game)

给定两个正整数 a 和 b（假设 $a \ge b$）。两人轮流操作，每次可以从大数中减去小数的任意正整数倍，即把 a 变为 $a - k \times b$（需保证操作后结果非负）。谁先得到 0 谁获胜。
若 $a \pmod{b} == 0$，先手必胜。
若 $a \ge 2b$，先手必胜。因为先手可以选择将状态转变为 $a \pmod{b}$ 或者 $(a \pmod{b}) + b$，其中必有一个是必胜态，先手只需把必败态丢给对手即可。
若 $b < a < 2b$，先手只能转移到 $a \pmod{b}$，胜负权发生交替，此时等价于普通的欧几里得算法（辗转相除）步数。

```
void solve(){
        scanf("%lld%lld",&a,&b);
        if(a<b)std::swap(a,b);
        bool win=true; // win 为 true 表示当前轮到的人能赢
        while(b){
                if(a>=2*b || a%b==0) break; // 满足条件当前玩家必胜
                a-=b; // 只能减去一次 b
                std::swap(a,b);
                win=!win; // 必胜权交替
        }
        if(win)puts("Stan wins"); // Stan 为先手
        else puts("Ollie wins"); // Ollie 为后手
}
```

Every-SG 游戏

规则：给定 n 个相互独立的 ICG。与普通 Nim 选择一个子游戏操作不同，在 Every-SG 中，对于**任何一个尚未结束的子游戏，玩家都必须对它进行一步合法操作**。任何一个子游戏无法操作时，该子游戏结束。所有子游戏都结束时，最后进行操作的玩家获胜。

核心结论：

由于所有能走的游戏都必须走，这就变成了一个关于**时间长短**的博弈。

如果某玩家在一个子游戏中处于必胜态，他一定会尽可能**拖延**该游戏的结束时间（让对手走最后一步无路可走）。

如果某玩家在一个子游戏中处于必败态，他一定会尽可能**缩短**该游戏的结束时间。

步数函数 $step(x)$ 计算：

我们需要计算状态 x 到游戏结束的步数 $step(x)$。

若 $SG(x)=0$（必败态）：$step(x)=\min\{step(y)\}+1$，其中 $x\to y$。

若 $SG(x)\neq 0$（必胜态）：$step(x)=\max\{step(y)\}+1$，其中 $x\to y$ 且 $SG(y)=0$。

最终判定：如果所有初始状态中，**最大的 $step$ 值是奇数**，则先手必胜；否则先手必败。

```cpp
#include<bits/stdc++.h>
const int N=10005;
int sg[N],step[N];
int vis[N],timer;
void get_sg_and_step(int x){
    if (sg[x]!=-1)return;
    // std::vector<int> next_states = get_transitions(x);
    // 这里为了演示，假设 next_states 已经存好
    std::vector<int> next_states;
    if (next_states.empty()){
        sg[x]=0;
        step[x]=0;
        return;
    }
    timer++;
    for (int y : next_states){
        get_sg_and_step(y);
        vis[sg[y]]=timer;
    }
    int res=0;
    while (vis[res]==timer)res++;
    sg[x]=res;

    // 计算 step
    if (sg[x]==0){ // 必败态，求 min
        int min_step=1e9;
        for (int y : next_states){
            min_step=std::min(min_step,step[y]);
        }
        step[x]=min_step+1;
    }else{         // 必胜态，在所有必败后继中求 max
        int max_step=0;
        for (int y : next_states){
            if (sg[y]==0){
                max_step=std::max(max_step,step[y]);
            }
        }
        step[x]=max_step+1;
    }
}
int n;
void solve(){
    scanf("%d",&n);
    int max_step=0;
    for (int i=1;i<=n;i++){
        int x;
        scanf("%d",&x);
        get_sg_and_step(x);
        max_step=std::max(max_step,step[x]);
    }
    // 判定 Every-SG：最大步数为奇数则先手胜
```

```
    if (max_step%2!=0)puts("First");
    else puts("Second");
}
int main(){
    std::memset(sg,-1,sizeof(sg));
    int t=1;
    scanf("%d",&t);
    while (t--){
        solve();
    }
    return 0;
}
```

二分图博弈

给定一张二分图，有一个棋子初始放在节点 s 上。两名玩家轮流移动棋子，每次只能沿着边将棋子移动到一个**未访问过**的相邻节点。无法移动棋子者判负。
先手必胜 $\iff$ 起点 s 一定在二分图的所有最大匹配中。

```cpp
#include<bits/stdc++.h>

const int N=20005;
const int M=200005;
int head[N],to[M],nxt[M],cnt;
int match[N];
bool vis[N],not_always[N];
int n,m,e;

void add_edge(int u,int v){
    to[++cnt]=v;nxt[cnt]=head[u];head[u]=cnt;
}

// 匈牙利算法找增广路
bool dfs(int u){
    for (int i=head[u];i;i=nxt[i]){
        int v=to[i];
        if (vis[v])continue;
        vis[v]=true;
        if (!match[v] || dfs(match[v])){
            match[v]=u;
            match[u]=v;
            return true;
        }
    }
    return false;
}

// 寻找交错树，标记不在所有最大匹配中的点
void mark_dfs(int u){
    not_always[u]=true;
    for (int i=head[u];i;i=nxt[i]){
        int v=to[i];
        // v 是对立侧的点，match[v] 是与 u 同侧的匹配点
        if (match[v] && !not_always[match[v]]){
            not_always[match[v]]=true;
            mark_dfs(match[v]);
        }
    }
}

void solve(){
    scanf("%d%d%d",&n,&m,&e);
    cnt=0;
    for (int i=1;i<=n+m;i++){
        head[i]=match[i]=0;
        not_always[i]=false;
    }

    for (int i=1;i<=e;i++){
        int u,v;
```

```
        scanf("%d%d",&u,&v);
        add_edge(u,v+n); // 右部点偏移 n 存储
        add_edge(v+n,u);
    }

    // 1. 求初始最大匹配
    int max_match=0;
    for (int i=1;i<=n;i++){
        if (!match[i]){
            for (int j=1;j<=n+m;j++) vis[j]=false;
            if (dfs(i)) max_match++;
        }
    }

    // 2. 从未匹配点出发跑交错树，标记非必须点
    for (int i=1;i<=n;i++){
        if (!match[i] && !not_always[i]) mark_dfs(i);
    }
    for (int i=n+1;i<=n+m;i++){
        if (!match[i] && !not_always[i]) mark_dfs(i);
    }

    // 3. 处理询问
    int q;
    scanf("%d",&q);
    while (q--){
        int start_node;
        scanf("%d",&start_node); // 注意：如果查询的是右部点，传入前可能需要 +n
        if (not_always[start_node]) puts("Second"); // 不在所有最大匹配中，后手必胜
        else puts("First");                         // 必定在所有最大匹配中，先手必胜
    }
}

int main(){
    int t=1;
    // scanf("%d",&t);
    while (t--){
        solve();
    }
    return 0;
}
```

Nim 游戏异或和计数

有 n 堆石子，每堆石子的数量可以从集合 S（例如 $1 \dots m$）中任意挑选。问有多少种挑选方案，使得先手必胜（或必败）？

先手必胜 $\iff$ 异或和不为 0。通常**正难则反**，先求异或和为 0（必败态）的方案数，再用总方案数减去它。 本质上是求多项式 $A(x) = \sum_{i \in S} x^i$ 在**异或卷积**意义下的 n 次方：$A(x)^n$。

1. 矩阵树定理 (Matrix-Tree Theorem)

这个定理的作用是**求一张图的生成树个数**。

核心构造：基尔霍夫矩阵 (Kirchhoff Matrix)

无论是无向图还是有向图，核心都是构造一个 $n \times n$ 的矩阵 L（Laplacian Matrix）：

$$L = D - A$$

- D **(度数矩阵)**：对角线矩阵。
- A **(邻接矩阵)**：记录点与点之间的边数（或边权和）。

具体规则：

1. **无向图**：
   - $D_{i,i}$ 为点 i 的度数。
   - $A_{i,j}$ 为点 i 和 j 之间的边数。
2. **有向图（内向树，指向根）**：
   - $D_{i,i}$ 为点 i 的**出度**。
   - $A_{i,j}$ 为点 i 到 j 的有向边数。
3. **有向图（外向树，由根向外）**：
   - $D_{i,i}$ 为点 i 的**入度**。
   - $A_{i,j}$ 为点 i 到 j 的有向边数。

定理结论

构造好 L 后，**删去第 k 行和第 k 列**（如果求有向图以 r 为根的生成树，就必须删去第 r 行和第 r 列），得到一个 $(n-1) \times (n-1)$ 的矩阵 L'。
生成树的个数，就等于矩阵 L' 的行列式 $\det(L')$。

高斯消元求行列式模板

在模质数意义下，利用高斯消元求行列式，时间复杂度 $O(n^3)$。

C++

```cpp
#include<bits/stdc++.h>

const int MOD=998244353;
const int N=305;
int a[N][N];
int n,m;

int qpow(int base,int exp){
    int res=1;
    while(exp){
        if(exp&1)res=1LL*res*base%MOD;
        base=1LL*base*base%MOD;
        exp>>=1;
    }
    return res;
}

// 求 (n-1)x(n-1) 矩阵的行列式
int det(int n){
    int res=1, sign=1;
    for (int i=1;i<=n;i++){
        int pivot=i;
        for (int j=i;j<=n;j++){
            if (a[j][i]){
                pivot=j;
                break;
            }
        }
        if (i!=pivot){
            std::swap(a[i],a[pivot]);
            sign=-sign; // 交换两行，行列式变号
        }
        if (!a[i][i]) return 0;

        res=1LL*res*a[i][i]%MOD;
        int inv=qpow(a[i][i],MOD-2);

        for (int j=i+1;j<=n;j++){
            if (a[j][i]){
                int rate=1LL*a[j][i]*inv%MOD;
                for (int k=i;k<=n;k++){
                    a[j][k]=(a[j][k]-1LL*rate*a[i][k]%MOD+MOD)%MOD;
                }
            }
        }
    }
    return (sign*res%MOD+MOD)%MOD;
}

void solve(){
    scanf("%d%d",&n,&m); // n 个点，m 条边
```

```
    std::memset(a,0,sizeof(a));
    for (int i=1;i<=m;i++){
        int u,v;
        scanf("%d%d",&u,&v);
        // 无向图构造 D - A
        a[u][u]++; a[v][v]++;
        a[u][v]--; a[v][u]--;
    }

    // 取模处理负数
    for (int i=1;i<=n;i++){
        for (int j=1;j<=n;j++){
            a[i][j]=(a[i][j]%MOD+MOD)%MOD;
        }
    }

    // 删去第 n 行第 n 列，求 n-1 阶行列式
    printf("%d\n",det(n-1));
}

int main(){
    int t=1;
    // scanf("%d",&t);
    while (t--){
        solve();
    }
    return 0;
}
```

2. 群论计数 (Burnside 引理 & Pólya 定理)

这俩兄弟专门处理“本质不同”的染色/排列问题（比如旋转、翻转后算同一种情况）。

Burnside 引理 (宏观：基于状态)

$$|X/G| = \frac{1}{|G|}\sum_{g\in G}|X^g|$$

- $|X/G|$：本质不同的方案数。
- $|G|$：置换群的大小（操作的种类数，比如旋转 n 次，翻转 n 次，共 $2n$ 种）。
- $|X^g|$：在置换操作 g 下，**保持不变的染色方案数**。
- **适用场景**：颜色数量受限，或者每种颜色的数量被固定时（只能用 Burnside 暴力搜索固定点）。

Pólya 定理 (微观：基于循环)

Pólya 是 Burnside 的特化版本，当你可以用 m 种颜色**不受限制**地去染 n 个位置时使用。

$$|X/G| = \frac{1}{|G|}\sum_{g\in G} m^{c(g)}$$

- $c(g)$：置换 g 拆分出的**循环节个数**。
- **适用场景**：无脑染色。只要算出每种操作有多少个循环，直接套公式。

经典模型：n 颗珠子染 m 种颜色的项链

只考虑旋转的情况下，n 种旋转操作中，每次旋转 i 步，产生的循环个数为 $\gcd(i,n)$。
方案数：

$$Ans=\frac{1}{n}\sum_{i=1}^{n}m^{\gcd(i,n)}$$

这里可以直接用欧拉函数优化到 $O(\sqrt{n})$（结合你的数论底子非常轻松）。

C++

```cpp
#include<bits/stdc++.h>

const int MOD=1e9+7;

int qpow(int a,int b){
    int res=1;
    while(b){
        if(b&1)res=1LL*res*a%MOD;
        a=1LL*a*a%MOD;
        b>>=1;
    }
    return res;
}

int get_phi(int n){
    int res=n;
    for (int i=2;i*i<=n;i++){
        if (n%i==0){
            res=res/i*(i-1);
            while(n%i==0)n/=i;
        }
    }
    if (n>1)res=res/n*(n-1);
    return res;
}

int n,m;
void solve(){
    scanf("%d%d",&n,&m); // n 颗珠子，m 种颜色
    if (n==0){
        puts("0");
        return;
    }
    int ans=0;
    // 枚举 gcd(i, n) 的值 d
    for (int d=1;d*d<=n;d++){
        if (n%d==0){
            ans=(ans+1LL*get_phi(n/d)*qpow(m,d)%MOD)%MOD;
            if (d*d!=n){
                ans=(ans+1LL*get_phi(d)*qpow(m,n/d)%MOD)%MOD;
            }
        }
    }
    ans=1LL*ans*qpow(n,MOD-2)%MOD;
    printf("%d\n",ans);
}

int main(){
    int t=1;
    scanf("%d",&t);
    while (t--){
```

```
        solve();
    }
    return 0;
}
```

3. Min-Max 容斥 (最值转化)

这是处理概率与期望中“收集齐所有物品所需期望时间”的终极核武器。

核心公式

对于任意集合 S，其最大值和最小值之间有如下转换关系：

$$\max(S) = \sum_{T \subseteq S, T \neq \emptyset} (-1)^{|T|-1} \min(T)$$

$$\min(S) = \sum_{T \subseteq S, T \neq \emptyset} (-1)^{|T|-1} \max(T)$$

在期望意义下依然完全成立：

$$E(\max(S)) = \sum_{T \subseteq S, T \neq \emptyset} (-1)^{|T|-1} E(\min(T))$$

赛场实战意义

- $E(\max(S))$：表示集合 S 中**最后一个**事件发生的期望时间（例如：抽卡游戏中，抽齐所有限定角色的期望次数）。这个极难直接算。
- $E(\min(T))$：表示子集 T 中**任何一个**事件第一次发生的期望时间。这个极好算！如果抽中 T 中任意一个角色的概率是 $P(T)$，那么期望次数就是几何分布的均值：

$$E(\min(T)) = \frac{1}{P(T)}$$

做题套路：

1. 遇到“求全集发生的最晚期望时间”。
2. 写出 Min-Max 容斥公式。
3. 对于所有的子集 T，算出发生任意一个的概率 $P(T)$，代入 $\frac{1}{P(T)}$。
4. 通常 n 比较小（$n \leq 20$），直接状态压缩枚举 T；如果 n 大，通常配合 FWT 或者 DP 来求那个带 $(-1)^{|T|-1}$ 符号的子集和。

普通线段树

```
struct Seg{
#define ls(x) (x<<1)
#define rs(x) (x<<1|1)
    ll tr[N<<2],tag[N<<2];
    void push_up(int p){
        tr[p]=tr[ls(p)]+tr[rs(p)];
    }
    void upd(int p,int k,int len){
        tr[p]+=1ll*k*len;
        tag[p]+=k;
    }
    void push_down(int p,int l,int r){
        if (tag[p]){
            int mid=(l+r)>>1;
            upd(ls(p),tag[p],mid-l+1);
            upd(rs(p),tag[p],r-mid);
            tag[p]=0;
        }
    }
    void build(int p,int l,int r,ll *a){
        tag[p]=0;
        if (l==r){
            tr[p]=1ll*a[l];return;
        }
        int mid=(l+r)>>1;
        build(ls(p),l,mid,a);
        build(rs(p),mid+1,r,a);
        push_up(p);
    }
    void insert(int p,int l,int r,int L,int R,int k){
        if (L<=l&&r<=R)return upd(p,k,r-l+1),void();
        int mid=(l+r)>>1;
        push_down(p,l,r);
        if (L<=mid)insert(ls(p),l,mid,L,R,k);
        if (R>mid)insert(rs(p),mid+1,r,L,R,k);
        push_up(p);
    }
    ll qry(int p,int l,int r,int L,int R){
        if (L<=l&&r<=R) return tr[p];
        int mid=(l+r)>>1;
        push_down(p,l,r);
        ll res=0;
        if (L<=mid)res+=qry(ls(p),l,mid,L,R);
        if (R>mid)res+=qry(rs(p),mid+1,r,L,R);
        return res;
    }
#undef ls
#undef rs
}T;
```

动态开点线段树

```
struct Seg{
    struct node{
        int ls,rs;
        ll tag,val;
    }tr[N*60];
    int tot=0;
#define ls(p) (tr[p].ls)
#define rs(p) (tr[p].rs)
#define tag(p) (tr[p].tag)
#define val(p) (tr[p].val)
    void push_up(int p){
        val(p)=val(ls(p))+val(rs(p));
    }
    void upd(int &p,ll k,int len){
        if (!p)p=++tot;
        val(p)+=1ull*len*k;
        tag(p)+=1ull*k;
    }
    void push_down(int p,int l,int r){
        if (tag(p)){
            int mid=(l+r)>>1;
            upd(ls(p),tag(p),mid-l+1);
            upd(rs(p),tag(p),r-mid);
            tag(p)=0;
        }
    }
    void insert(int &p,int l,int r,int L,int R,int k){
        if (!p)p=++tot;
        if (L<=l&&r<=R){
            upd(p,1ull*k,r-l+1);return ;
        }
        int mid=(l+r)>>1;
        push_down(p,l,r);
        if (L<=mid)insert(ls(p),l,mid,L,R,k);
        if (R>mid)insert(rs(p),mid+1,r,L,R,k);
        push_up(p);
    }
    ll qry(int p,int l,int r,int L,int R){
        if (!p)return 0;
        if (L<=l&&r<=R){return val(p);}
        int mid=(l+r)>>1;
        push_down(p,l,r);
        ll res=0;
        if (L<=mid)res+=qry(ls(p),l,mid,L,R);
        if (R>mid)res+=qry(rs(p),mid+1,r,L,R);
        return res;
    }
#undef ls
#undef rs
#undef tag
```

```
#undef val
}T;
```

线段树合并

```
struct SegTree{
    struct node{
        int ls,rs;
        ll ct;
    }tr[N*50];
    int cnt=0;
#define ls(x) (tr[x].ls)
#define rs(x) (tr[x].rs)
#define ct(x) (tr[x].ct)
    void push_up(int p){
        ct(p)=ct(ls(p))+ct(rs(p));
    }
    void insert(int &p,int l,int r,int x,int k){
        if (!p)p=++cnt;
        if (l==r)return ct(p)+=k,void();
        int mid=l+r>>1;
        if (x<=mid)insert(ls(p),l,mid,x,k);
        else insert(rs(p),mid+1,r,x,k);
        push_up(p);
    }
    void merge(int &u,int v,int l,int r){
        if (!u||!v)return u+=v,void();
        int mid=l+r>>1;
        if(l==r){ct(u)=ct(u)+ct(v);return;}
        merge(ls(u),ls(v),l,mid);
        merge(rs(u),rs(v),mid+1,r);
        push_up(u);
    }
#undef ls
#undef rs
#undef ct
}T;
```

李超线段树

维护平面线段求最值，封装了直线的求值逻辑，支持区间插入。

```
struct SegTree{
#define ls(x) (x<<1)
#define rs(x) (x<<1|1)
    struct LINE{
        ll k,b;
        ll get(ll x){
            return k*x+b;
        }
    }tr[N<<2];
    int vis[N<<2];
    ll mi[N<<2];
    void init(int n){
        for (int i=1;i<4*n;i++)mi[i]=INF;
    }
    ll get_min(int p){
        return mi[p];
    }
    void upd(int p,int l,int r,LINE line,ll *a){
        if (!vis[p]){
            vis[p]=1;
            tr[p]=line;
            mi[p]=std::min(tr[p].get(a[l]),tr[p].get(a[r]));
            if (l<r)mi[p]=std::min(mi[p],std::min(mi[ls(p)],mi[rs(p)]));
            return;
        }
        int mid=(l+r)>>1;
        ll l1=line.get(a[l]),r1=line.get(a[r]);
        ll l2=tr[p].get(a[l]),r2=tr[p].get(a[r]);
        if (l1>=l2&&r1>=r2)return;
        if (l1<=l2&&r1<=r2){
            tr[p]=line;mi[p]=std::min(l1,r1);
            if (l<r)mi[p]=std::min(mi[p],std::min(get_min(ls(p)),get_min(rs(p))));
            return;
        }
        if (tr[p].get(a[mid])>line.get(a[mid]))std::swap(tr[p],line);
        if (l==r)return mi[p]=tr[p].get(a[l]),void();
        if (tr[p].get(a[l])>line.get(a[l]))upd(ls(p),l,mid,line,a);
        else if (tr[p].get(a[r])>line.get(a[r]))upd(rs(p),mid+1,r,line,a);
        mi[p]=std::min(tr[p].get(a[l]),tr[p].get(a[r]));
        mi[p]=std::min(mi[p],std::min(get_min(ls(p)),get_min(rs(p))));
    }
    void insert(int p,int l,int r,int L,int R,LINE line,ll *a){
        if (L<=l&&r<=R)return upd(p,l,r,line,a),void();
        int mid=(l+r)>>1;
        if (L<=mid)insert(ls(p),l,mid,L,R,line,a);
        if (R>mid)insert(rs(p),mid+1,r,L,R,line,a);
        mi[p]=vis[p]?std::min(tr[p].get(a[l]),tr[p].get(a[r])):INF;
        if (l<r)mi[p]=std::min(mi[p],std::min(get_min(ls(p)),get_min(rs(p))));
    }
    ll qry(int p,int l,int r,int L,int R,ll* a){
        if (L<=l&&r<=R)return get_min(p);
```

```
        int mid=(l+r)>>1;
        ll res=INF;
        if (vis[p])res=std::min(res,
                            std::min(tr[p].get(a[std::max(l,L)]),tr[p].get(a[std::min(r,R)])));
        if (L<=mid)res=std::min(res,qry(ls(p),l,mid,L,R,a));
        if (R>mid)res=std::min(res,qry(rs(p),mid+1,r,L,R,a));
        return res;
    }
#undef ls
#undef rs
}T;
```

线段树分治

```
struct edge{
    int u,v;
    bitset<1001> w;
};
struct SegTree{
#define ls(x) (x<<1)
#define rs(x) (x<<1|1)
    vector<edge>tr[N<<2];
    void insert(int p,int l,int r,int L,int R,edge w){
        if (L<=l&&r<=R)return tr[p].push_back(w),void();
        int mid=(l+r)>>1;
        if (L<=mid)insert(ls(p),l,mid,L,R,w);
        if (R>mid)insert(rs(p),mid+1,r,L,R,w);
    }
    void solve(int p,int l,int r,Basis b){
        int bottom=dsu.top;
        for (auto [u,v,w]:tr[p]){
            if (!dsu.merge(u,v,w)){
                auto [fu,val_u]=dsu.find(u);
                auto [fv,val_v]=dsu.find(v);
                b.insert(val_u^val_v^w);
            }
        }
        if (l==r){
            print(b.qry(0));
        }
        else{
            int mid=l+r>>1;
            solve(ls(p),l,mid,b);
            solve(rs(p),mid+1,r,b);
        }
        dsu.del(bottom);
    }
#undef ls
#undef rs
}T;
```

可持久化线段树（主席树）

```
struct ChairmanTree{
    struct node{
        int ls,rs,cnt;
    }tr[N*40];
    int rt[N],tot=0;
#define ls(x) (tr[x].ls)
#define rs(x) (tr[x].rs)
#define cnt(x) (tr[x].cnt)
    void insert(int &p,int pre,int l,int r,int val){
        p=++tot;
        tr[p]=tr[pre];
        cnt(p)++;
        if (l==r)return;
        int mid=(l+r)>>1;
        if (val<=mid)insert(ls(p),ls(pre),l,mid,val);
        else insert(rs(p),rs(pre),mid+1,r,val);
    }
    int qry(int p,int pre,int l,int r,int k){
        if (l==r)return l;
        int mid=(l+r)>>1;
        int sum=cnt(ls(p))-cnt(ls(pre));
        if (k<=sum)return qry(ls(p),ls(pre),l,mid,k);
        else return qry(rs(p),rs(pre),mid+1,r,k-sum);
    }
#undef ls
#undef rs
#undef cnt
}T;
// 使用：T.insert(rt[i], rt[i-1], 1, len, a[i]);
```

点分治

```
int siz[N],mx[N],vis[N],rt,sum_node;
using std::vector;
struct edge{int v,w;};
vector<edge> g[N];

void get_rt(int u,int fa){
    siz[u]=1;mx[u]=0;
    for (auto [v,w]:g[u]){
        if (v==fa||vis[v])continue;
        get_rt(v,u);
        siz[u]+=siz[v];
        mx[u]=std::max(mx[u],siz[v]);
    }
    mx[u]=std::max(mx[u],sum_node-siz[u]);
    if (mx[u]<mx[rt])rt=u;
}
void get_dis(int u,int fa,int dis,vector<int>&vec){
    vec.push_back(dis);
    for (auto [v,w]:g[u]){
        if (v==fa||vis[v])continue;
        get_dis(v,u,dis+w,vec);
    }
}
void calc(int u){
    // 处理以 u 为重心的路径，通常结合双指针或树状数组统计
}
void solve_tree(int u){
    vis[u]=1;
    calc(u);
    for (auto [v,w]:g[u]){
        if (vis[v])continue;
        sum_node=siz[v];
        rt=0;mx[0]=INF;
        get_rt(v,0);
        solve_tree(rt);
    }
}
// 主函数调用：sum_node=n; rt=0; mx[0]=INF; get_rt(1,0); solve_tree(rt);
```

Kruskal 重构树

讲解：在 Kruskal 求最小生成树的过程中，每次合并两个连通块时，不直接连边，而是新建一个虚拟节点，权值为这条边的边权，然后把这个虚拟节点作为两个连通块根节点的父亲。
性质：原图两点之间所有路径中，最大边权的最小值，就等于重构树上这两点 LCA 的节点权值。

```cpp
struct KruskalTree{
    int fa[N<<1],val[N<<1],tot;
    int ls[N<<1],rs[N<<1]; // 左右儿子
    int find(int x){return fa[x]==x?x:fa[x]=find(fa[x]);}
    void build(int n,int m,vector<pair<int,pair<int,int>>>&edges){
        for (int i=1;i<=n*2;i++)fa[i]=i;
        std::sort(edges.begin(),edges.end()); // 按边权排序
        tot=n;
        for (auto e:edges){
            int w=e.first;
            int u=e.second.first,v=e.second.second;
            int fu=find(u),fv=find(v);
            if (fu!=fv){
                val[++tot]=w;
                fa[fu]=fa[fv]=tot;
                ls[tot]=fu; rs[tot]=fv;
            }
        }
    }
}KTree;
// 建成后，对这棵树(根为tot)跑 LCA 即可
```

基础分块

```
int bl[N],L[N],R[N],tag[N];
ll sum[N],a[N];
int block_size,num;

void build(int n){
    block_size=sqrt(n);
    num=n/block_size+(n%block_size?1:0);
    for (int i=1;i<=num;i++){
        L[i]=(i-1)*block_size+1;
        R[i]=std::min(i*block_size,n);
        for (int j=L[i];j<=R[i];j++){
            bl[j]=i;
            sum[i]+=a[j];
        }
    }
}
void upd(int l,int r,ll k){
    int p=bl[l],q=bl[r];
    if (p==q){
        for (int i=l;i<=r;i++)a[i]+=k,sum[p]+=k;
        return;
    }
    for (int i=l;i<=R[p];i++)a[i]+=k,sum[p]+=k;
    for (int i=p+1;i<=q-1;i++)tag[i]+=k,sum[i]+=1ll*(R[i]-L[i]+1)*k;
    for (int i=L[q];i<=r;i++)a[i]+=k,sum[q]+=k;
}
ll qry(int l,int r){
    int p=bl[l],q=bl[r];
    ll res=0;
    if (p==q){
        for (int i=l;i<=r;i++)res+=a[i]+tag[p];
        return res;
    }
    for (int i=l;i<=R[p];i++)res+=a[i]+tag[p];
    for (int i=p+1;i<=q-1;i++)res+=sum[i];
    for (int i=L[q];i<=r;i++)res+=a[i]+tag[q];
    return res;
}
```

吉如一线段树

```
struct SegTree{
    struct node{
        ll sum;
        int max,max_cnt,second_max,maxb;
        int tag1,tag2,tag3,tag4;
    }tr[N<<2];
#define ls(p) (p<<1)
#define rs(p) (p<<1|1)
#define sum(p) (tr[p].sum)
#define mx(p) (tr[p].max)
#define ct(p) (tr[p].max_cnt)
#define se(p) (tr[p].second_max)
#define mb(p) (tr[p].maxb)
#define t1(p) (tr[p].tag1)
#define t2(p) (tr[p].tag2)
#define t3(p) (tr[p].tag3)
#define t4(p) (tr[p].tag4)
    void push_up(int p){
        sum(p)=sum(ls(p))+sum(rs(p));
        mx(p)=std::max(mx(ls(p)),mx(rs(p)));
        mb(p)=std::max(mb(ls(p)),mb(rs(p)));
        if (mx(ls(p))==mx(rs(p))){
            ct(p)=ct(ls(p))+ct(rs(p));
            se(p)=std::max(se(ls(p)),se(rs(p)));
        }
        else if (mx(ls(p))>mx(rs(p))){
            ct(p)=ct(ls(p));
            se(p)=std::max(se(ls(p)),mx(rs(p)));
        }
        else{
            ct(p)=ct(rs(p));
            se(p)=std::max(mx(ls(p)),se(rs(p)));
        }
    }
    void build(int p,int l,int r,int *a){
        t1(p)=t2(p)=t3(p)=t4(p)=0;
        if (l==r){
            sum(p)=1ll*a[l];
            mx(p)=mb(p)=a[l];
            ct(p)=1;se(p)=-2e9;
            return;
        }
        int mid=(l+r)>>1;
        build(ls(p),l,mid,a);
        build(rs(p),mid+1,r,a);
        push_up(p);
    }
    void upd_add(int p,int len,int k){
        sum(p)+=1ll*ct(p)*k+1ll*(len-ct(p))*k;
        mx(p)+=k;
```

```
    mb(p)=std::max(mb(p),mx(p));
    if (se(p)!=-2e9)se(p)+=k;
    t1(p)+=k;t2(p)+=k;
    t3(p)=std::max(t1(p),t3(p));
    t4(p)=std::max(t2(p),t4(p));
}
void upd_min(int p,int v){
    int k=mx(p)-v;
    sum(p)-=1ll*ct(p)*k;
    mx(p)=v;
    t1(p)-=k;
}
void upd_tag(int p,int len,int a1,int a2,int a3,int a4){
    sum(p)+=1ll*ct(p)*a1+1ll*(len-ct(p))*a2;
    mb(p)=std::max(mb(p),mx(p)+a3);
    mx(p)+=a1;
    if (se(p)!=-2e9)se(p)+=a2;
    t3(p)=std::max(t3(p),t1(p)+a3);
    t4(p)=std::max(t4(p),t2(p)+a4);
    t1(p)+=a1;t2(p)+=a2;
}
void push_down(int p,int l,int r){
    int MAX=std::max(mx(ls(p)),mx(rs(p)));
    int mid=(l+r)>>1;
    if (MAX==mx(ls(p))){
        upd_tag(ls(p),mid-l+1,t1(p),t2(p),t3(p),t4(p));
    }
    else upd_tag(ls(p),mid-l+1,t2(p),t2(p),t4(p),t4(p));
    if (MAX==mx(rs(p))){
        upd_tag(rs(p),r-mid,t1(p),t2(p),t3(p),t4(p));
    }
    else upd_tag(rs(p),r-mid,t2(p),t2(p),t4(p),t4(p));
    t1(p)=t2(p)=t3(p)=t4(p)=0;
}
void insert_add(int p,int l,int r,int L,int R,int k){
    if (L<=l&&r<=R)return upd_add(p,r-l+1,k),void();
    int mid=(l+r)>>1;
    push_down(p,l,r);
    if (L<=mid)insert_add(ls(p),l,mid,L,R,k);
    if (R>mid)insert_add(rs(p),mid+1,r,L,R,k);
    push_up(p);
}
void insert_min(int p,int l,int r,int L,int R,int k){
    if (k>=mx(p))return;
    if (L<=l&&r<=R&&se(p)<k)return upd_min(p,k),void();
    int mid=(l+r)>>1;
    push_down(p,l,r);
    if (L<=mid)insert_min(ls(p),l,mid,L,R,k);
    if (R>mid)insert_min(rs(p),mid+1,r,L,R,k);
    push_up(p);
}
ll qry_sum(int p,int l,int r,int L,int R){
```

```
        if (L<=l&&r<=R)return sum(p);
        int mid=(l+r)>>1;
        push_down(p,l,r);
        ll res=0;
        if (L<=mid)res+=qry_sum(ls(p),l,mid,L,R);
        if (R>mid)res+=qry_sum(rs(p),mid+1,r,L,R);
        return res;
    }
    int qry_mx(int p,int l,int r,int L,int R){
        if (L<=l&&r<=R)return mx(p);
        int mid=(l+r)>>1;
        push_down(p,l,r);
        int res=-2e9;
        if (L<=mid)res=std::max(res,qry_mx(ls(p),l,mid,L,R));
        if (R>mid)res=std::max(res,qry_mx(rs(p),mid+1,r,L,R));
        return res;
    }
    int qry_mb(int p,int l,int r,int L,int R){
        if (L<=l&&r<=R)return mb(p);
        int mid=(l+r)>>1;
        push_down(p,l,r);
        int res=-2e9;
        if (L<=mid)res=std::max(res,qry_mb(ls(p),l,mid,L,R));
        if (R>mid)res=std::max(res,qry_mb(rs(p),mid+1,r,L,R));
        return res;
    }
#undef ls
#undef rs
#undef sum
#undef mx
#undef ct
#undef se
#undef mb
#undef t1
#undef t2
#undef t3
#undef t4
}T;
```

可持久化 01 Trie

```
struct PersistentTrie{
    struct node{
        int son[2],cnt;
    }tr[N*32];
    int rt[N],tot=0;
#define son(x,i) (tr[x].son[i])
#define cnt(x) (tr[x].cnt)

    // 插入数值 x，基于上一个版本 pre
    void insert(int &p,int pre,int val){
        p=++tot;
        tr[p]=tr[pre];
        cnt(p)++;
        int u=p,v=pre;
        for (int i=30;i>=0;i--){
            int c=(val>>i)&1;
            son(u,c)=++tot;
            tr[son(u,c)]=tr[son(v,c)];
            u=son(u,c);
            v=son(v,c);
            cnt(u)++;
        }
    }

    // 在版本 [L-1, R] 的 Trie 中查询 val 与区间内某个数异或的最大值
    int qry(int L_rt,int R_rt,int val){
        int u=L_rt,v=R_rt;
        int res=0;
        for (int i=30;i>=0;i--){
            int c=(val>>i)&1;
            // 贪心：走与 c 相反的方向以获得最大的异或位
            if (cnt(son(v,1^c))-cnt(son(u,1^c))>0){
                res|=(1<<i);
                u=son(u,1^c);
                v=son(v,1^c);
            }
            else{
                u=son(u,c);
                v=son(v,c);
            }
        }
        return res;
    }
#undef son
#undef cnt
}PT;
```

可持久化并查集

支持并查集的**历史版本查询**以及**历史版本回退/基于某历史版本的修改**

```
struct PersistentDSU{
    struct node{
        int ls,rs,fa;
    }tr[N*40];
    int rt[N],tot=0;
#define ls(x) (tr[x].ls)
#define rs(x) (tr[x].rs)
#define fa(x) (tr[x].fa)

    // 建树初始化每个点的父亲和初始深度（这里把深度/秩作为附加权值或单独建树，通常一并维护）
    void build(int &p,int l,int r){
        p=++tot;
        if (l==r)return fa(p)=l,void();
        int mid=(l+r)>>1;
        build(ls(p),l,mid);
        build(rs(p),mid+1,r);
    }

    // 单点修改（持久化更新某个节点的父亲）
    void insert(int &p,int pre,int l,int r,int x,int f){
        p=++tot;
        tr[p]=tr[pre];
        if (l==r)return fa(p)=f,void();
        int mid=(l+r)>>1;
        if (x<=mid)insert(ls(p),ls(pre),l,mid,x,f);
        else insert(rs(p),rs(pre),mid+1,r,x,f);
    }

    // 单点查询（在版本 p 中查节点 x 的父亲）
    int query(int p,int l,int r,int x){
        if (l==r)return fa(p);
        int mid=(l+r)>>1;
        if (x<=mid)return query(ls(p),l,mid,x);
        else return query(rs(p),mid+1,r,x);
    }

    // 不带路径压缩的查找根节点（需要配合可持久化深度/秩查询）
    int find(int p,int x){
        int f=query(p,1,n,x);
        if (f==x)return x;
        return find(p,f);
    }
#undef ls
#undef rs
#undef fa
}PDSU;
```

KMP / 前缀函数 (Prefix Function)

- **单模式串匹配**：在文本串中寻找模式串出现的位置和次数。
- **求字符串的最短整周期**：对于长度为 n 的字符串，如果 $n \bmod (n - pi[n-1]) == 0$，那么最短循环节长度就是 $n - pi[n-1]$。
- **Border 树（失配树）**： `pi[i]` 指向的是当前前缀的最长真前缀和真后缀相等的长度。在一些复杂的字符串计数问题中，通过不断跳 `pi` 数组，可以遍历一个前缀的所有 Border。

```
int n,m,pi[N];
using std::string;
string s1,s2;

void pre(string s){
    int len=s.length();
    pi[0]=0;
    for (int i=1;i<len;i++){
        int j=pi[i-1];
        while(j&&s[i]!=s[j])j=pi[j-1];
        pi[i]=j+(s[i]==s[j]);
    }
}
void solve(){
    // pre(s2+"#"+s1); s2为模式串，s1为文本串
    pre(s2+"#"+s1);
    for (int i=0;i<n;i++){
        if (pi[m+1+i]==m)printf("%d\n",i+2-m);
    }
    for (int i=0;i<m;i++){
        printf("%d ",pi[i]);
    }
    puts("");
}
```

AC 自动机 (Aho-Corasick Automaton)

- **多模式串匹配**：同时查询多个单词在长文本中的出现次数。
- **AC 自动机 + DP（极高频考点）**：这是区域赛的高频考点。通常状态设计为 `dp[i][j]`，表示当前生成了长度为 i 的字符串，且在 AC 自动机上的节点 j 时的方案数 / 最大权值。通过 AC 自动机上转移边建立转移矩阵，经常还能配合**矩阵快速幂**进行优化。

```
using std::string;
using std::queue;

namespace ACAM {
    struct tire{
        int son[26],fail,in,id,ans;
        void clear(){
            fail=in=id=ans=0;
            memset(son,0,sizeof(son));
        }
    }T[N];
    int cntid,cntn,ans[N];
    queue<int>Q;

    void init(){
        cntid=cntn=0;
        T[0].clear();
    }
    int insert(string s){
        int u=0,len=s.length();
        for (int i=0;i<len;i++){
            int& son=T[u].son[s[i]-'a'];
            if (!son)son=++cntn,T[son].clear();
            u=son;
        }
        if (!T[u].id)T[u].id=++cntid;
        return T[u].id;
    }
    void build(){
        while (!Q.empty()) Q.pop();
        for (int i=0;i<26;i++){
            if (T[0].son[i])Q.push(T[0].son[i]);
        }
        while (!Q.empty()){
            int u=Q.front();Q.pop();
            for (int i=0;i<26;i++){
                if (T[u].son[i]){
                    T[T[u].son[i]].fail=T[T[u].fail].son[i];
                    T[T[T[u].fail].son[i]].in++;
                    Q.push(T[u].son[i]);
                }
                else {
                    T[u].son[i]=T[T[u].fail].son[i];
                }
            }
        }
    }
    void qry(string t){
        int u=0,len=t.length();
        for (int i=0;i<len;i++){
            u=T[u].son[t[i]-'a'];
```

```
            T[u].ans++;
        }
    }
    void topo(){
        while (!Q.empty()) Q.pop();
        for (int i=0;i<=cntn;i++){
            if (!T[i].in)Q.push(i);
        }
        while (!Q.empty()){
            int u=Q.front();Q.pop();
            ans[T[u].id]=T[u].ans;
            int v=T[u].fail;
            T[v].ans+=T[u].ans;
            if (!--T[v].in)Q.push(v);
        }
    }
}
```

后缀自动机 (Suffix Automaton, SAM)

- **不同子串计数**：每个节点包含的子串数量是 `len(p) - len(link(p))`，求和即可。
- **多串最长公共子串**：将一个串建 SAM，其它串在上面跑匹配。
- **结合线段树合并**：在 SAM 的 Parent 树（Link 树）上进行线段树合并，可以维护每个子串在原串中的 `endpos`（出现位置集合），从而解决极其复杂的子串定位和计数问题。

```
struct SAM {
    struct Node {
        int len, fa;
        int ch[26];
    } st[N << 1];
    int sz = 1, last = 1;

    void extend(int c) {
        int cur = ++sz;
        st[cur].len = st[last].len + 1;
        int p = last;
        while (p && !st[p].ch[c]) {
            st[p].ch[c] = cur;
            p = st[p].fa;
        }
        if (!p) {
            st[cur].fa = 1;
        } else {
            int q = st[p].ch[c];
            if (st[p].len + 1 == st[q].len) {
                st[cur].fa = q;
            } else {
                int clone = ++sz;
                st[clone] = st[q];
                st[clone].len = st[p].len + 1;
                while (p && st[p].ch[c] == q) {
                    st[p].ch[c] = clone;
                    p = st[p].fa;
                }
                st[q].fa = st[cur].fa = clone;
            }
        }
        last = cur;
    }
} sam;
```

1. KMP：本质是“失配指针”与“周期”

不要把 KMP 只当成字符串匹配工具，它的核心产物只有那个 `pi` 数组（也叫 `nxt` 或 `fail` 数组）。

核心物理意义：

`pi[i]` 的值，就是原串前缀 `S[0...i]` 中，**最长的、且相等的“真前缀”和“真后缀”的长度**。

赛场怎么用？

- **用法一：找周期（极其高频）**
  - **结论**：对于一个长度为 n 的字符串，它的最短循环节长度是 $n - pi[n-1]$。
  - **判定**：如果 $n \bmod (n - pi[n-1]) == 0$ 且 $pi[n-1] > 0$，说明这个串是完美的周期串，循环了 $n/(n - pi[n-1])$ 次。在很多要求字符串折叠、求最小周期的构造题中直接套用。
- **用法二：Border 树（失配树）**

- **操作**：把每个 i 和 $pi[i]$ 连一条有向边（$pi[i] \to i$），会形成一棵树。
- **意义**：在这棵树上，从根节点到 i 的路径上的所有节点，全都是 `S[0...i]` 的合法 Border（即既是前缀又是后缀的子串）。如果题目要求统计前缀在各个位置出现的次数，直接在这棵树上跑树形 DP（子树求和）就行。

2. AC 自动机 (ACAM)：本质是“Trie 图” + “DP 转移矩阵”

你的模板里最后那步 `T[u].son[i] = T[T[u].fail].son[i]` 把 Trie 树补全成了一个**图**（Trie 图）。这是 ACAM 最强大的地方。

核心物理意义：

ACAM 上的每一个节点，代表了一个**状态**。当你在这个图上顺着边往下走，走到节点 u 时，意味着你当前生成的字符串的**最长后缀**，匹配到了某个模式串的前缀。

赛场怎么用？

- **用法一：多模式串词频统计**
  - **操作**：把你手里的长文本（母串）放在 Trie 图上从根开始跑，跑到节点 u 就 `T[u].ans++`（打个标记）。跑完之后，顺着 `fail` 指针的拓扑序，把 `ans` 往父节点累加（你的模板里的 `topo()` 函数干的就是这个事）。
- **用法二：AC 自动机上 DP（金牌区绝对主力）**
  - **场景**：题目不给你母串，而是让你**构造**一个长度为 N 的母串，要求包含某些模式串（或者不包含），求方案数或最大得分。
  - **做法**：设 `dp[i][u]` 表示当前生成了长度为 i 的串，现在停在 AC 自动机的节点 u 上的方案数。
  - **转移**：枚举下一个填的字母 c（`0` 到 `25`），下一个状态就是 $v = T[u].son[c]$。那么 `dp[i+1][v] = (dp[i+1][v] + dp[i][u])`。如果 N 很大（比如 10^{18}），把这个转移关系写成矩阵，直接跑**矩阵快速幂**。

3. 后缀自动机 (SAM)：本质是“全子串压缩图” + “线段树合并”

SAM 听起来吓人，但实战中你只需要关注它建出来的两样东西：**节点** 和 **Parent 树（Link 树）**。

核心物理意义：

- **节点**：每个节点代表**一类**子串。这类子串在原串中出现的结束位置（`endpos` 集合）是完全一模一样的。
- **Link 树（父指针）**：每个节点的 `link`（或 `fa`）指向上一个节点，这构成了一棵树。子节点代表的子串，永远是父节点代表的子串的**加长版后缀**。

赛场怎么用？

- **用法一：不同子串计数/长度统计**
  - 每个节点 u 包含的本质不同的子串个数刚好是 $len[u] - len[fa[u]]$。求整个串有多少不同子串，直接把所有节点的这个差值加起来即可。
- **用法二：结合线段树合并（最契合你的科技树）**
  - **场景**：题目问你某个子串在区间 $[L, R]$ 里出现了多少次。
  - **做法**：SAM 的每次 `extend` 时，给新建的节点对应的**权值线段树**在当前长度的位置插入一个 1。建完 SAM 后，在 Parent 树上跑**线段树合并**（子节点向父节点合并）。
  - 合并完之后，每个节点上的线段树就完美记录了这个节点代表的所有子串在原串中出现的所有位置。你只需要倍增找到查询子串对应的 SAM 节点，然后直接在这个节点的线段树里查询 $[L, R]$ 区间的和就行了。这刚好用得上你扎实的线段树合并底子。

马拉车算法 (Manacher's Algorithm) 是专门用来在 $O(n)$ 时间内解决**回文子串**问题的极其巧妙的算法。

在字符串问题中，回文串最大的麻烦在于**长度有奇偶之分**（奇数长度中心是一个字符，偶数长度中心在两个字符之间）。马拉车的核心操作只有一个：**强行插入特殊字符，把所有回文串全部变成奇数长度**。

1. 本质与物理意义

马拉车算法的核心产物是 **回文半径数组 `p[i]`**。

- **预处理**：把原串 `s = "abba"`，在首尾和每个字符中间插上 `#`，变成 `t = "$#a#b#b#a#^"`（首尾加 `$` 和 `^` 是为了防止越界，省去边界判断）。
- **物理意义**：`p[i]` 表示在处理后的字符串 `t` 中，以 `t[i]` 为中心的最长回文串的**半径**（包含中心点）。
- **最神仙的结论**：`p[i] - 1`，**刚好就是这个回文串在原字符串 `s` 中的实际长度**！

2. 怎么用

马拉车在比赛中绝不是只能用来求“最长回文子串”，它通常用来做这三件事：

- **用法一：求最长回文子串**
  - 跑完算法后，直接遍历 `p` 数组，求 `max(p[i]) - 1` 即可。
- **用法二：统计本质不一定不同的回文子串总数**
  - 以 `t[i]` 为中心的回文串，在原串中会贡献 `p[i] / 2` 个合法的回文子串（向下取整）。
  - 把所有的 `p[i] / 2` 累加起来，就是原串中所有回文子串的总数。
- **用法三：$O(1)$ 极速判断任意区间是否为回文串**
  - 很多复杂的 DP 题或贪心题，需要频繁判断原串 `S[L...R]` 是不是回文串。如果是用字符串 Hash 容易被卡或者有常数问题，马拉车是完美的 $O(1)$ 判定工具。
  - **做法**：原串中区间 $[L, R]$ 的中心位置，映射到新串 `t` 中的下标是 `mid = L + R + 2`（假设原串下标从 0 开始）。只需要判断 `p[mid] - 1 >= R - L + 1`，如果成立，原串的 $[L, R]$ 就是绝对的回文串。

C++

```cpp
int p[N << 1]; // 注意数组开两倍大
using std::string;
using std::min;

void manacher(string s) {
    string t = "$#";
    for (char c : s) {
        t += c;
        t += '#';
    }
    t += '^'; // 首尾加不同字符防止 while 越界，极其优雅的 trick

    int len = t.length();
    int C = 0, R = 0;

    for (int i = 1; i < len - 1; i++) {
        // 利用对称性，快速初始化 p[i]
        p[i] = (i < R) ? min(R - i, p[2 * C - i]) : 1;

        // 暴力向外扩展，因为有首尾特殊字符，不会越界
        while (t[i + p[i]] == t[i - p[i]]) p[i]++;

        // 更新最右边界 R 和对应的中心 C
        if (i + p[i] > R) {
            R = i + p[i];
            C = i;
        }
    }
}
```

1. CDQ 分治 (CDQ Divide and Conquer)

本质与核心逻辑：

CDQ 分治的核心思想是**降维**。它用来处理多维偏序问题（比如最经典的“三维偏序/陌上花开”）。

- **第一维**：直接排序。
- **第二维**：利用分治，在归并排序的过程中统计左半部分对右半部分的贡献。
- **第三维**：借助基础数据结构（通常是树状数组）来维护。

物理意义：

把动态的“修改+查询”问题，转化为静态的“左区间的修改，对右区间的查询产生的贡献”。只要操作满足交换律和结合律，且允许离线，几乎都可以用 CDQ 分治把 $O(N^2)$ 的暴力降到 $O(N \log^2 N)$。

三维偏序模板：

C++

```cpp
#include<bits/stdc++.h>
#define N 100050
#define K 200050

struct node{
    int a,b,c,cnt,ans;
    bool operator!=(const node& other)const{
        return a!=other.a||b!=other.b||c!=other.c;
    }
}q[N],tmp[N];

bool cmp1(const node& x,const node& y){
    if (x.a!=y.a)return x.a<y.a;
    if (x.b!=y.b)return x.b<y.b;
    return x.c<y.c;
}
bool cmp2(const node& x,const node& y){
    if (x.b!=y.b)return x.b<y.b;
    return x.c<y.c;
}

struct BIT{
    int tr[K],limit;
#define lowbit(x) (x&-x)
    void add(int x,int v){
        for(;x<=limit;x+=lowbit(x))tr[x]+=v;
    }
    int qry(int x){
        int res=0;
        for(;x;x-=lowbit(x))res+=tr[x];
        return res;
    }
    void clear(int x){
        for(;x<=limit;x+=lowbit(x))tr[x]=0;
    }
#undef lowbit
}T;

int n,k,ans_cnt[N],tot=0;

void cdq(int l,int r){
    if (l==r)return;
    int mid=(l+r)>>1;
    cdq(l,mid);
    cdq(mid+1,r);

    int i=l,j=mid+1,p=l;
    while (i<=mid&&j<=r){
        if (q[i].b<=q[j].b){
            T.add(q[i].c,q[i].cnt);
            tmp[p++]=q[i++];
```

```
        }
        else{
            q[j].ans+=T.qry(q[j].c);
            tmp[p++]=q[j++];
        }
    }
    while (i<=mid)tmp[p++]=q[i++];
    while (j<=r){
        q[j].ans+=T.qry(q[j].c);
        tmp[p++]=q[j++];
    }
    for (int k=l;k<=mid;k++)T.clear(q[k].c);
    for (int k=l;k<=r;k++)q[k]=tmp[k];
}

void solve(){
    // 读取原始数据到 tmp，然后去重合并到 q 数组，记录 cnt
    // 此处省略读取，假设去重后的有效元素个数为 tot
    std::sort(q+1,q+tot+1,cmp1);
    cdq(1,tot);
    for (int i=1;i<=tot;i++){
        ans_cnt[q[i].ans+q[i].cnt-1]+=q[i].cnt;
    }
    for (int i=0;i<n;i++)printf("%d\n",ans_cnt[i]);
}
```

2. 整体二分 (Parallel Binary Search)

本质与核心逻辑：

如果一个问题只有一个询问，你可以二分答案然后用数据结构 `check` 它是否合法。但如果有 Q 个询问，每个都二分会超时。

整体二分将**所有询问放在一起二分**答案的值域 $[L, R]$。每次取 mid，将所有修改和查询操作依次执行，通过判断查询结果与 mid 的关系，将操作划分为两部分：

- **左半边**：答案在 $[L, mid]$ 的询问，以及值 $\leq mid$ 的修改。
- **右半边**：答案在 $(mid, R]$ 的询问，以及值 $> mid$ 的修改。

物理意义：

用时间换空间，把原本需要“可持久化权值线段树（主席树）”或“树套树”来维护的多维静态/动态第 k 大问题，转化为离线对值域的分治。代码极其好写，常数极小。

动态区间第 k 小模板（带修莫队的劲敌）：

C++

```cpp
#include<bits/stdc++.h>
#define N 200050
#define INF 0x3f3f3f3f

struct Query{
    int type; // 1: 修改, 2: 查询
    int x,y,k,id; // 修改时 x 位置加/减 y; 查询时区间 [x,y] 找第 k 小, id为查询编号
}q[N*3],q1[N*3],q2[N*3];

int n,m,a[N],ans[N],tot=0,cnt_qry=0;

struct BIT{
    int tr[N];
#define lowbit(x) (x&-x)
    void add(int x,int v){
        for(;x<=n;x+=lowbit(x))tr[x]+=v;
    }
    int qry(int x){
        int res=0;
        for(;x;x-=lowbit(x))res+=tr[x];
        return res;
    }
#undef lowbit
}T;

void solve(int l,int r,int ql,int qr){
    if (ql>qr)return;
    if (l==r){
        for (int i=ql;i<=qr;i++){
            if (q[i].type==2)ans[q[i].id]=l;
        }
        return;
    }
    int mid=(l+r)>>1;
    int p1=0,p2=0;

    for (int i=ql;i<=qr;i++){
        if (q[i].type==1){
            if (q[i].y<=mid){
                T.add(q[i].x,q[i].k); // k 存放的是 +1 或 -1
                q1[++p1]=q[i];
            }
            else{
                q2[++p2]=q[i];
            }
        }
        else{
            int sum=T.qry(q[i].y)-T.qry(q[i].x-1);
            if (q[i].k<=sum){
                q1[++p1]=q[i];
            }
```

```
            else{
                q[i].k-=sum; // 扣除左半边的贡献，去右半边找
                q2[++p2]=q[i];
            }
        }
    }

    // 撤销树状数组上的修改，保持数据结构干净
    for (int i=1;i<=p1;i++){
        if (q1[i].type==1)T.add(q1[i].x,-q1[i].k);
    }

    for (int i=1;i<=p1;i++)q[ql+i-1]=q1[i];
    for (int i=1;i<=p2;i++)q[ql+p1+i-1]=q2[i];

    solve(l,mid,ql,ql+p1-1);
    solve(mid+1,r,ql+p1,qr);
}

void main_logic(){
    // 读入操作，把初始数组 a[i] 当成在 i 位置插入 a[i] 的操作 (type=1, x=i, y=a[i], k=1)
    // 把修改操作拆成两次：删除原来的数 (k=-1)，插入新的数 (k=1)
    // 记录查询总数 cnt_qry
    solve(-INF,INF,1,tot); // 值域区间 [-INF, INF]
    for (int i=1;i<=cnt_qry;i++){
        printf("%d\n",ans[i]);
    }
}
```